Business Case

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

The company wants to understand and process the data coming out of data engineering pipelines:

- Clean, sanitize and manipulate data to get useful features out of raw fields
- Make sense out of the raw data and help the data science team to build forecasting models on it

Columns Description

- data - tells whether the data is testing or training data
- trip_creation_time: Timestamp of trip creation
- route_schedule_uuid: Unique Id for a particular route schedule
- route_type: Transportation type
- FTL(Full Truck Load): FTL shipments get to the destination sooner, as the truck is making no other pickups or drop-offs along the way
- Carting: Handling system consisting of small vehicles (carts) trip_uuid - Unique ID given to a particular trip (A trip may include different source and destination centers)
- source_center - Source ID of trip origin
- source_name - Source Name of trip origin
- destination_cente: Destination ID
- destination_name: Destination Name
- od_start_time: Trip start time
- od_end_time : Trip end time
- start_scan_to_end_scan: Time taken to deliver from source to destination
- is_cutoff: Unknown field
- cutoff_factor: Unknown field
- cutoff_timestamp: Unknown field
- actual_distance_to_destination: Distance in Kms between source and destination warehouse
- actual_time: Actual time taken to complete the delivery (Cumulative)
- osrm_time: An open-source routing engine time calculator which computes the shortest path between points in a given map (Includes usual traffic, distance through major and minor roads) and gives the time (Cumulative)
- osrm_distance: An open-source routing engine which computes the shortest path between points in a given map (Includes usual traffic, distance through major and minor roads) (Cumulative) factor
- segment_actual_time: This is a segment time. Time taken by the subset of the package delivery
- segment_osrm_time: This is the OSRM segment time. Time taken by the subset of the package delivery
- segment_osrm_distance: This is the OSRM distance. Distance covered by subset of the package delivery
- segment_factor: Unknown field

Data Loading & Importing Libraries

In [43]:
```
# Importing libraries

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind, ttest_rel, mannwhitneyu, kruskal, shapiro, f_oneway, chi2_contingency,levene
```

In [44]:
```
# Importing data from google drive

df = pd.read_csv('/content/drive/MyDrive/1. Rahul/Training/Scalar_Data_Science&ML/4.Business_Case_Study/delhivery_data.csv')
```

Data Summarization

In [45]:
```
df.head()
```

Out[45]:

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_sta
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	201 03:21:32
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	201 03:21:32
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	201 03:21:32
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	201 03:21:32
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	201 03:21:32

In [46]:
```
# Number of rows and columns

print('Number of rows:{}'.format(df.shape[0]))
print('Number of columns:{}'.format(df.shape[1]))
```

```
Number of rows:144867
Number of columns:24
```

In [47]:
```
# Name of the columns
df.columns
```

Out[47]:
```
Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type',
       'trip_uuid', 'source_center', 'source_name', 'destination_center',
       'destination_name', 'od_start_time', 'od_end_time',
       'start_scan_to_end_scan', 'is_cutoff', 'cutoff_factor',
       'cutoff_timestamp', 'actual_distance_to_destination', 'actual_time',
       'osrm_time', 'osrm_distance', 'factor', 'segment_actual_time',
       'segment_osrm_time', 'segment_osrm_distance', 'segment_factor'],
      dtype='object')
```

In [48]:
```
# Data types of the columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144867 non-null  object
 1   trip_creation_time              144867 non-null  object
 2   route_schedule_uuid             144867 non-null  object
 3   route_type                      144867 non-null  object
 4   trip_uuid                       144867 non-null  object
 5   source_center                   144867 non-null  object
 6   source_name                     144574 non-null  object
 7   destination_center              144867 non-null  object
 8   destination_name                144606 non-null  object
 9   od_start_time                   144867 non-null  object
 10  od_end_time                     144867 non-null  object
 11  start_scan_to_end_scan          144867 non-null  float64
 12  is_cutoff                       144867 non-null  bool
 13  cutoff_factor                   144867 non-null  int64
 14  cutoff_timestamp                144867 non-null  object
 15  actual_distance_to_destination  144867 non-null  float64
 16  actual_time                     144867 non-null  float64
 17  osrm_time                       144867 non-null  float64
 18  osrm_distance                   144867 non-null  float64
 19  factor                          144867 non-null  float64
 20  segment_actual_time             144867 non-null  float64
 21  segment_osrm_time               144867 non-null  float64
 22  segment_osrm_distance           144867 non-null  float64
 23  segment_factor                  144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
```

In [49]:
```
# We can note that there are few date time columns, so we will convert it into date time format for better analysis
# - trip_creation_time
# - od_start_time
# - od_end_time
# - cutoff_timestamp
# - actual_time
# - osrm_time
# - segment_actual_time
# - segment_osrm_time
```

In [50]:
```
# Listing down all the columns whose data type needs to be changed

columns_to_convert = ['trip_creation_time','od_start_time','od_end_time','cutoff_timestamp']
```

In [51]:
```
# Then applying apply function to convert it into datetype data type

for x in columns_to_convert:
  df[x] = pd.to_datetime(df[x], errors='coerce', format='mixed')
```

In [52]:
```
# Lets recheck the data types

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144867 non-null  object
 1   trip_creation_time              144867 non-null  datetime64[ns]
 2   route_schedule_uuid             144867 non-null  object
 3   route_type                      144867 non-null  object
 4   trip_uuid                       144867 non-null  object
 5   source_center                   144867 non-null  object
 6   source_name                     144574 non-null  object
 7   destination_center              144867 non-null  object
 8   destination_name                144606 non-null  object
 9   od_start_time                   144867 non-null  datetime64[ns]
 10  od_end_time                     144867 non-null  datetime64[ns]
 11  start_scan_to_end_scan          144867 non-null  float64
 12  is_cutoff                       144867 non-null  bool
 13  cutoff_factor                   144867 non-null  int64
 14  cutoff_timestamp                144867 non-null  datetime64[ns]
 15  actual_distance_to_destination  144867 non-null  float64
 16  actual_time                     144867 non-null  float64
 17  osrm_time                       144867 non-null  float64
 18  osrm_distance                   144867 non-null  float64
 19  factor                          144867 non-null  float64
 20  segment_actual_time             144867 non-null  float64
 21  segment_osrm_time               144867 non-null  float64
 22  segment_osrm_distance           144867 non-null  float64
 23  segment_factor                  144867 non-null  float64
dtypes: bool(1), datetime64[ns](4), float64(10), int64(1), object(8)
memory usage: 25.6+ MB
```

In [53]:
```
# As we can see that all the columns represeting date are now
# converted into datetype data type
```

In [54]:
```
# Renaming the columns for ease of reading it

df.rename(columns = {'start_scan_to_end_scan':'duration_source_dest'},inplace = True)
```

In [55]:
```
# Statistical Summary of the data
df.describe(include = np.number)

# It will provide the summary of only numeric columns
```

Out[55]:

	duration_source_dest	cutoff_factor	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	factor	segment_actual_time	segment_osr
count	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000	144867.
mean	961.262986	232.926567	234.073372	416.927527	213.868272	284.771297	2.120107	36.196111	18.
std	1037.012769	344.755577	344.990009	598.103621	308.011085	421.119294	1.715421	53.571158	14
min	20.000000	9.000000	9.000045	9.000000	6.000000	9.008200	0.144000	-244.000000	0.
25%	161.000000	22.000000	23.355874	51.000000	27.000000	29.914700	1.604264	20.000000	11.
50%	449.000000	66.000000	66.126571	132.000000	64.000000	78.525800	1.857143	29.000000	17.
75%	1634.000000	286.000000	286.708875	513.000000	257.000000	343.193250	2.213483	40.000000	22.
max	7898.000000	1927.000000	1927.447705	4532.000000	1686.000000	2326.199100	77.387097	3051.000000	1611.

In [56]:
```
# # Inferences:
# - average duration is 961 with median of 449 and has a maximum value of 7898 which might suggest outliers which might skew the data towards
# right or positive skewed
# - There is a minro varation between the actual distance vs osrm distance, which the later on the higher side, need to understand the causes
```

Data Pre-processing and Feature Engineering

In [57]:
```
# Since we have multiple rows for the same trip may be because it was divided into multiple segments
# hence would be aggregating the columns either using first, sum, mean, min, max depending upon the
# features or column
```

Grouping the data

In [58]:
```
# Aggregating the data

pd.options.display.max_columns = None
modified_df = df.groupby(
    ['trip_uuid', 'source_center', 'destination_center']).agg({
    'data': 'first',
    'route_schedule_uuid': 'first',
    'route_type': 'first',
    'source_name': 'first',
    'destination_name': 'first',
    'trip_creation_time': 'min',
    'od_start_time': 'min',
    'od_end_time': 'max',
    'duration_source_dest': 'max',
    'is_cutoff': 'any', # True if any segment in the leg was cut off
    'cutoff_factor': 'mean',
    'cutoff_timestamp': 'max', # Latest cutoff timestamp for the leg
    'actual_distance_to_destination': 'max',
    'actual_time': 'max',
    'osrm_time': 'max',
    'osrm_distance': 'max',
    'factor': 'mean', # Assuming 'factor' is a ratio similar to segment_factor
    'segment_actual_time': 'sum',
    'segment_osrm_time': 'sum',
    'segment_osrm_distance': 'sum',
    'segment_factor': 'mean'
}).reset_index()
```

In [59]:
```
modified_df.head()
```

Out[59]:

	trip_uuid	source_center	destination_center	data	route_schedule_uuid	route_type	source_name	destination_name	trip_creation_time
0	trip-153671041653548748	IND209304AAA	IND000000ACB	training	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	Kanpur_Central_H_6 (Uttar Pradesh)	Gurgaon_Bilaspur_HB (Haryana)	2018-09-12 00:00:16.535741
1	trip-153671041653548748	IND462022AAA	IND209304AAA	training	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	Bhopal_Trnsport_H (Madhya Pradesh)	Kanpur_Central_H_6 (Uttar Pradesh)	2018-09-12 00:00:16.535741
2	trip-153671042288605164	IND561203AAB	IND562101AAA	training	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	Doddablpur_ChikaDPP_D (Karnataka)	Chikblapur_ShntiSgr_D (Karnataka)	2018-09-12 00:00:22.886430
3	trip-153671042288605164	IND572101AAA	IND561203AAB	training	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	Tumkur_Veersagr_I (Karnataka)	Doddablpur_ChikaDPP_D (Karnataka)	2018-09-12 00:00:22.886430
4	trip-153671043369099517	IND000000ACB	IND160002AAC	training	thanos::sroute:de5e208e-7641-45e6-8100-4d9fb1e...	FTL	Gurgaon_Bilaspur_HB (Haryana)	Chandigarh_Mehmdpur_H (Punjab)	2018-09-12 00:00:33.691250

Null value treatment

In [60]:
```
# Checking if there are any null values

modified_df.isnull().sum()
```

Out[60]:

	0
trip_uuid	0
source_center	0
destination_center	0
data	0
route_schedule_uuid	0
route_type	0
source_name	66
destination_name	81
trip_creation_time	0
od_start_time	0
od_end_time	0
duration_source_dest	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

In [61]:
```
# There are null values, lets check its proportion

(modified_df.isnull().sum() / modified_df.shape[0])*100
```

Out[61]:

	0
trip_uuid	0.000000
source_center	0.000000
destination_center	0.000000
data	0.000000
route_schedule_uuid	0.000000
route_type	0.000000
source_name	0.250303
destination_name	0.307191
trip_creation_time	0.000000
od_start_time	0.000000
od_end_time	0.000000
duration_source_dest	0.000000
is_cutoff	0.000000
cutoff_factor	0.000000
cutoff_timestamp	0.000000
actual_distance_to_destination	0.000000
actual_time	0.000000
osrm_time	0.000000
osrm_distance	0.000000
factor	0.000000
segment_actual_time	0.000000
segment_osrm_time	0.000000
segment_osrm_distance	0.000000
segment_factor	0.000000

dtype: float64

In [62]:
```
# Since the source_name and destination_name are important field and its difficult to substitute it hence the best decision is to
# drop them

# Dropping null values in both the columns


modified_df.dropna(subset = ['source_name','destination_name'],inplace = True)
```

In [63]:
```
# Rechecking the null values
(modified_df.isnull().sum() / modified_df.shape[0])*100
```

Out[63]:

	0
trip_uuid	0.0
source_center	0.0
destination_center	0.0
data	0.0
route_schedule_uuid	0.0
route_type	0.0
source_name	0.0
destination_name	0.0
trip_creation_time	0.0
od_start_time	0.0
od_end_time	0.0
duration_source_dest	0.0
is_cutoff	0.0
cutoff_factor	0.0
cutoff_timestamp	0.0
actual_distance_to_destination	0.0
actual_time	0.0
osrm_time	0.0
osrm_distance	0.0
factor	0.0
segment_actual_time	0.0
segment_osrm_time	0.0
segment_osrm_distance	0.0
segment_factor	0.0

dtype: float64

Dropping unwanted columns

In [64]:
```
# None of the columns have null values, also there are some columns for which we dont have complete visibility on its significance, hence
# would be droping them

# Columns which will be dropped:

# - is_cutoff
# - cutoff_factor
# - cutoff_timestamp
# - factor
# - segment_factor


modified_df.drop(columns = ['is_cutoff','cutoff_factor','cutoff_timestamp','factor','segment_factor'],inplace = True)
```

In [65]:
```
# Revised dataframe would be

modified_df.head()
```

Out[65]:

	trip_uuid	source_center	destination_center	data	route_schedule_uuid	route_type	source_name	destination_name	trip_creation_time
0	trip-153671041653548748	IND209304AAA	IND000000ACB	training	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	Kanpur_Central_H_6 (Uttar Pradesh)	Gurgaon_Bilaspur_HB (Haryana)	2018-09-12 00:00:16.535741
1	trip-153671041653548748	IND462022AAA	IND209304AAA	training	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	Bhopal_Trnsport_H (Madhya Pradesh)	Kanpur_Central_H_6 (Uttar Pradesh)	2018-09-12 00:00:16.535741
2	trip-153671042288605164	IND561203AAB	IND562101AAA	training	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	Doddablpur_ChikaDPP_D (Karnataka)	Chikblapur_ShntiSgr_D (Karnataka)	2018-09-12 00:00:22.886430
3	trip-153671042288605164	IND572101AAA	IND561203AAB	training	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	Tumkur_Veersagr_I (Karnataka)	Doddablpur_ChikaDPP_D (Karnataka)	2018-09-12 00:00:22.886430
4	trip-153671043369099517	IND000000ACB	IND160002AAC	training	thanos::sroute:de5e208e-7641-45e6-8100-4d9fb1e...	FTL	Gurgaon_Bilaspur_HB (Haryana)	Chandigarh_Mehmdpur_H (Punjab)	2018-09-12 00:00:33.691250

Splitting source and destination centre name and state

In [66]:
```
# Since we have source centre name and location in the same columns, hence splittig
# it into two columns

modified_df['source_centre_name'] = modified_df['source_name'].str.split('(',expand=True)[0]
modified_df['source_state'] = modified_df['source_name'].str.split('(',expand = True)[1].str.replace(')','')
```

In [67]:
```
# checking the values of both columns (Source)

modified_df[['source_centre_name','source_state']].head()
```

Out[67]:

	source_centre_name	source_state
0	Kanpur_Central_H_6	Uttar Pradesh
1	Bhopal_Trnsport_H	Madhya Pradesh
2	Doddablpur_ChikaDPP_D	Karnataka
3	Tumkur_Veersagr_I	Karnataka
4	Gurgaon_Bilaspur_HB	Haryana

In [68]:
```
# Similarly we will do for destination name

modified_df['destination_centre_name'] = modified_df['destination_name'].str.split('(',expand=True)[0]
modified_df['destination_state'] = modified_df['destination_name'].str.split('(',expand = True)[1].str.replace(')','')
```

In [69]:
```
# Since we have extracted the required information from source_name and destination_name, we will drop these columns
# Also there are additional columns which we dont required as route_schedule_uuid, source_center,destination_center,
# we will be removing these columns also.


modified_df.drop(columns=['source_name','destination_name','route_schedule_uuid','source_center','destination_center'],inplace = True)
```

Applying Time functions

In [70]:
```
# Extracting critical time related information from trip creation time field

modified_df['trip_creation_month'] = modified_df['trip_creation_time'].dt.month
modified_df['trip_creation_day'] = modified_df['trip_creation_time'].dt.day
modified_df['trip_creation_dayofweek'] = modified_df['trip_creation_time'].dt.dayofweek
modified_df['trip_creation_hour'] = modified_df['trip_creation_time'].dt.hour
```

In [71]:
```
# Revised dataframe looks like:

modified_df.head()
```

Out[71]:

	trip_uuid	data	route_type	trip_creation_time	od_start_time	od_end_time	duration_source_dest	actual_distance_to_destination	actual_time	osrm_tim
0	trip-153671041653548748	training	FTL	2018-09-12 00:00:16.535741	2018-09-12 16:39:46.858469	2018-09-13 13:40:23.123744	1260.0	383.759164	732.0	349
1	trip-153671041653548748	training	FTL	2018-09-12 00:00:16.535741	2018-09-12 00:00:16.535741	2018-09-12 16:39:46.858469	999.0	440.973689	830.0	394
2	trip-153671042288605164	training	Carting	2018-09-12 00:00:22.886430	2018-09-12 02:03:09.655591	2018-09-12 03:01:59.598855	58.0	24.644021	47.0	26
3	trip-153671042288605164	training	Carting	2018-09-12 00:00:22.886430	2018-09-12 00:00:22.886430	2018-09-12 02:03:09.655591	122.0	48.542890	96.0	42
4	trip-153671043369099517	training	FTL	2018-09-12 00:00:33.691250	2018-09-14 03:40:17.106733	2018-09-14 17:34:55.442454	834.0	242.309306	611.0	212

Calculate the time taken between od_start_time and od_end_time and keep it as a feature. Drop the original columns

In [82]:
```
# Adding an additional feature using values from the exisinting features

modified_df['od_duration'] = (modified_df['od_end_time'] - modified_df['od_start_time']).dt.total_seconds()/60
```

In [84]:
```
# Dropping the original columns

modified_df.drop(columns = ['od_end_time','od_start_time'],inplace = True)
```

In [85]:
```
# Revised dataframe:

modified_df.head()
```

Out[85]:

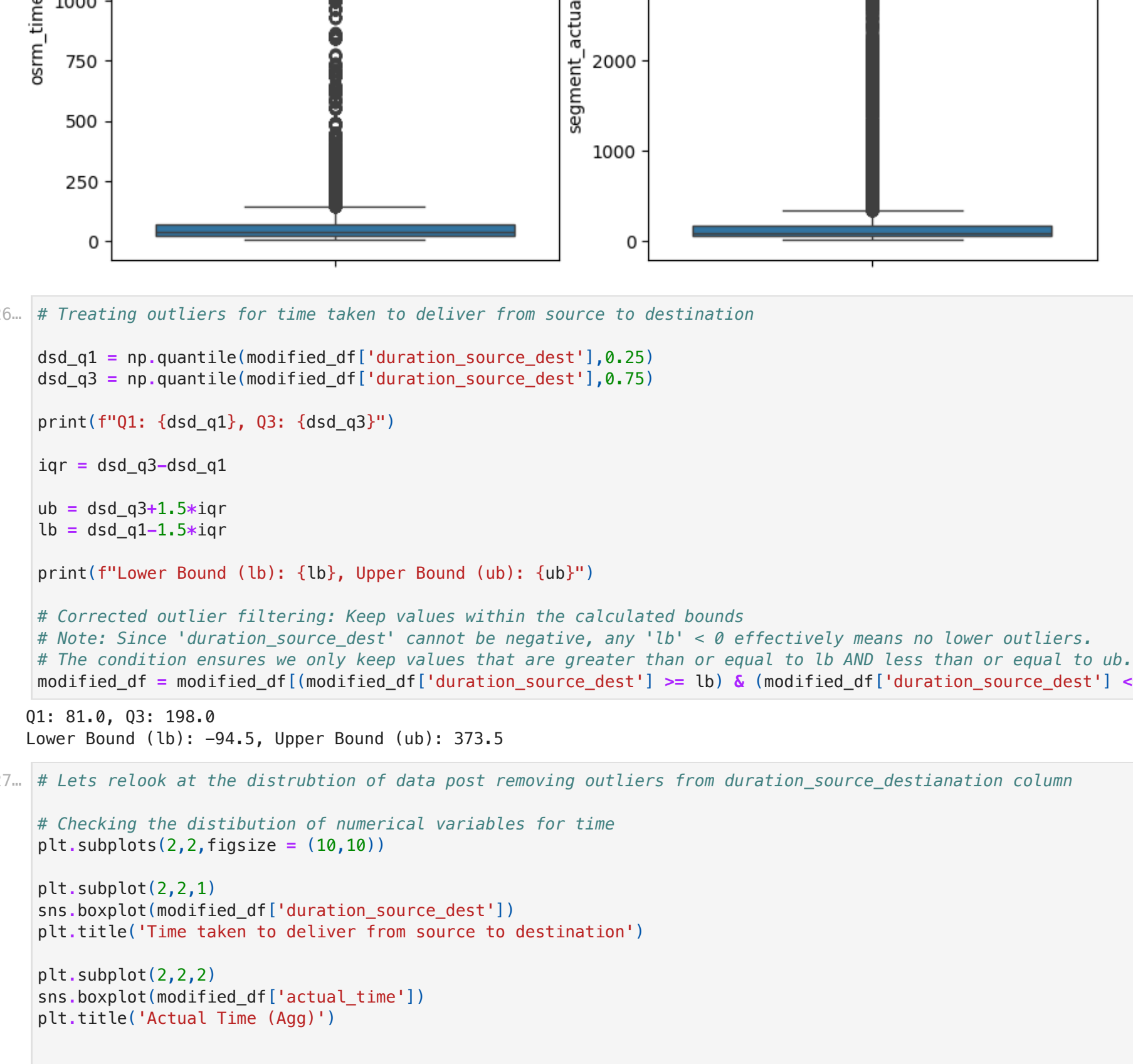
	trip_uuid	data	trip_creation_time	duration_source_dest	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segm
0	trip-153671041653548748	training	2018-09-12 00:00:16.535741	1260.0	383.759164	732.0	349.0	446.5496	728.0	
1	trip-153671041653548748	training	2018-09-12 00:00:16.535741	999.0	440.973689	830.0	394.0	544.8027	820.0	
2	trip-153671042288605164	training	2018-09-12 00:00:22.886430	58.0	24.644021	47.0	26.0	28.1994	46.0	
3	trip-153671042288605164	training	2018-09-12 00:00:22.886430	122.0	48.542890	96.0	42.0	56.9116	95.0	
4	trip-153671043369099517	training	2018-09-12 00:00:33.691250	834.0	242.309306	611.0	212.0	281.2109	608.0	

In [92]:
```
modified_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 26222 entries, 0 to 26367
Data columns (total 22 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   trip_uuid                       26222 non-null  object
 1   data                            26222 non-null  object
 2   trip_creation_time              26222 non-null  datetime64[ns]
 3   duration_source_dest            26222 non-null  float64
 4   actual_distance_to_destination  26222 non-null  float64
 5   actual_time                     26222 non-null  float64
 6   osrm_time                       26222 non-null  float64
 7   osrm_distance                   26222 non-null  float64
 8   segment_actual_time             26222 non-null  float64
 9   segment_osrm_time               26222 non-null  float64
 10  segment_osrm_distance           26222 non-null  float64
 11  source_centre_name              26222 non-null  object
 12  source_state                    26222 non-null  object
 13  destination_centre_name         26222 non-null  object
 14  destination_state               26222 non-null  object
 15  trip_creation_month             26222 non-null  int32
 16  trip_creation_day               26222 non-null  int32
 17  trip_creation_dayofweek         26222 non-null  int32
 18  trip_creation_hour              26222 non-null  int32
 19  Carting                         26222 non-null  int64
 20  FTL                             26222 non-null  int64
 21  od_duration                     26222 non-null  float64
dtypes: datetime64[ns](1), float64(9), int32(4), int64(2), object(6)
memory usage: 4.2+ MB
```

Outliers Treatment

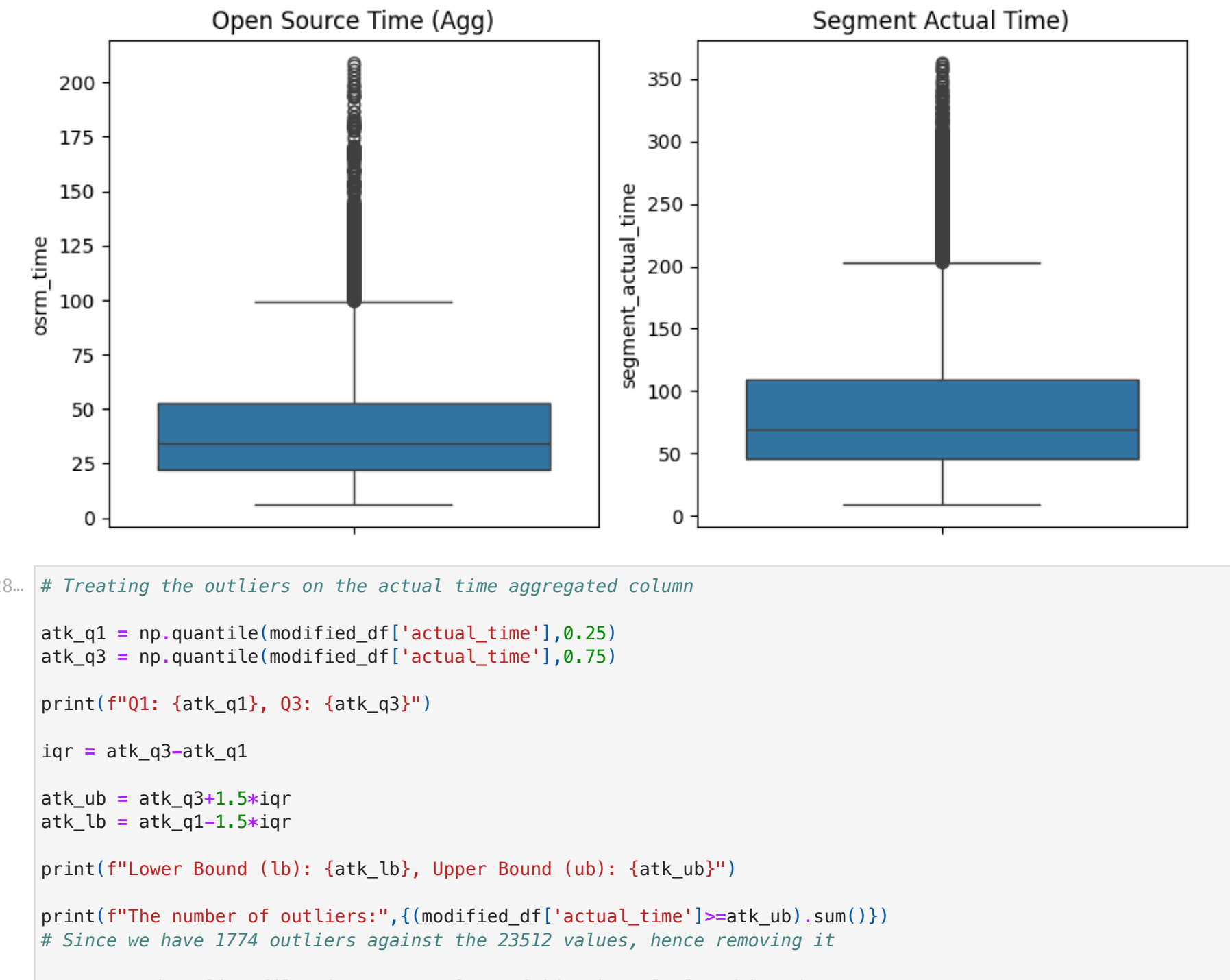
In [100…
```
# Checking the distibution of numerical variables for time
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['duration_source_dest'])
plt.title('Time taken to deliver from source to destination')

plt.subplot(2,2,2)
sns.boxplot(modified_df['actual_time'])
plt.title('Actual Time (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['osrm_time'])
plt.title('Open Source Time (Agg)')


plt.subplot(2,2,4)
sns.boxplot(modified_df['segment_actual_time'])
plt.title('Segment Actual Time)')

plt.show()
```



In [126…
```
# Treating outliers for time taken to deliver from source to destination

dsd_q1 = np.quantile(modified_df['duration_source_dest'],0.25)
dsd_q3 = np.quantile(modified_df['duration_source_dest'],0.75)

print(f"Q1: {dsd_q1}, Q3: {dsd_q3}")

iqr = dsd_q3-dsd_q1

ub = dsd_q3+1.5*iqr
lb = dsd_q1-1.5*iqr

print(f"Lower Bound (lb): {lb}, Upper Bound (ub): {ub}")

# Corrected outlier filtering: Keep values within the calculated bounds
# Note: Since 'duration_source_dest' cannot be negative, any 'lb' < 0 effectively means no lower outliers.
# The condition ensures we only keep values that are greater than or equal to lb AND less than or equal to ub.
modified_df = modified_df[(modified_df['duration_source_dest'] >= lb) & (modified_df['duration_source_dest'] <= ub)]
```

```
Q1: 81.0, Q3: 198.0
Lower Bound (lb): -94.5, Upper Bound (ub): 373.5
```

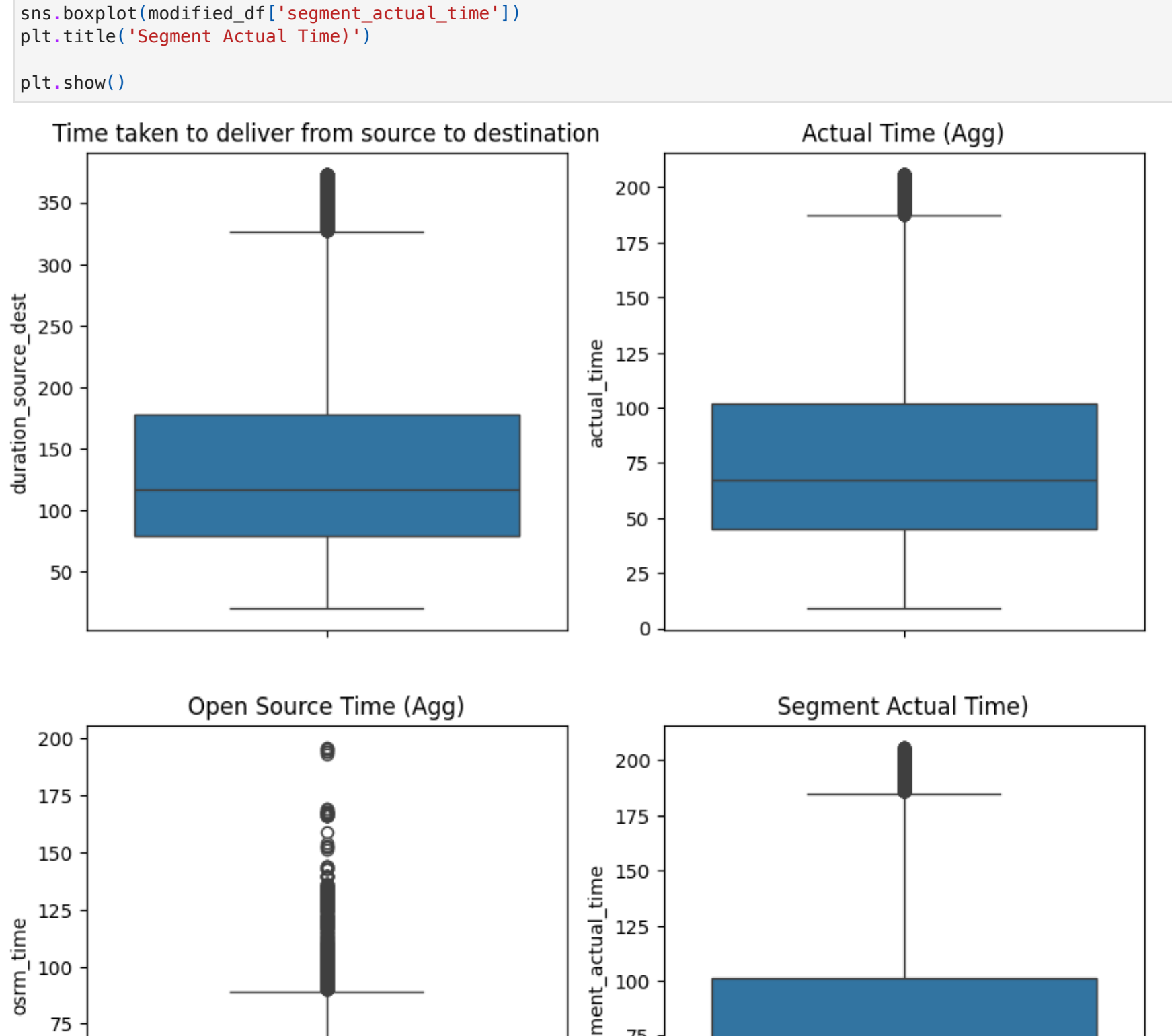
In [127…
```
# Lets relook at the distrubtion of data post removing outliers from duration_source_destianation column

# Checking the distibution of numerical variables for time
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['duration_source_dest'])
plt.title('Time taken to deliver from source to destination')

plt.subplot(2,2,2)
sns.boxplot(modified_df['actual_time'])
plt.title('Actual Time (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['osrm_time'])
plt.title('Open Source Time (Agg)')


plt.subplot(2,2,4)
sns.boxplot(modified_df['segment_actual_time'])
plt.title('Segment Actual Time)')

plt.show()
```



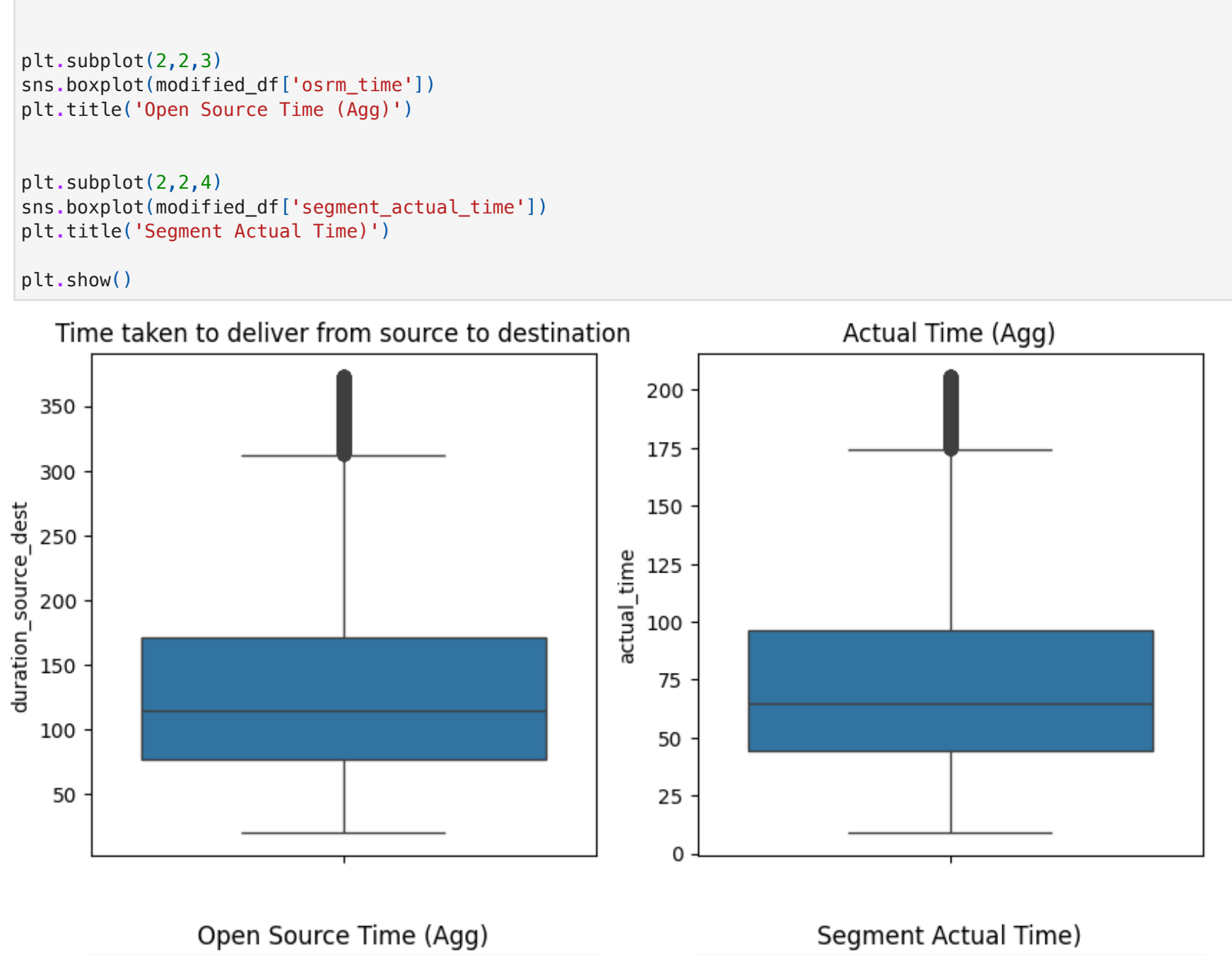
In [128…
```
# Treating the outliers on the actual time aggregated column

atk_q1 = np.quantile(modified_df['actual_time'],0.25)
atk_q3 = np.quantile(modified_df['actual_time'],0.75)

print(f"Q1: {atk_q1}, Q3: {atk_q3}")

iqr = atk_q3-atk_q1

atk_ub = atk_q3+1.5*iqr
atk_lb = atk_q1-1.5*iqr

print(f"Lower Bound (lb): {atk_lb}, Upper Bound (ub): {atk_ub}")

print(f"The number of outliers:",{(modified_df['actual_time']>=atk_ub).sum()})
# Since we have 1774 outliers against the 23512 values, hence removing it

# Corrected outlier filtering: Keep values within the calculated bounds
# Note: Since 'duration_source_dest' cannot be negative, any 'lb' < 0 effectively means no lower outliers.
# The condition ensures we only keep values that are greater than or equal to lb AND less than or equal to ub.
modified_df = modified_df[(modified_df['actual_time'] >= atk_lb) & (modified_df['actual_time'] <= atk_ub)]

print(f"The revised shape of the dataframe is:",{modified_df.shape})
```

```
Q1: 46.0, Q3: 110.0
Lower Bound (lb): -50.0, Upper Bound (ub): 206.0
The number of outliers: {np.int64(1092)}
The revised shape of the dataframe is: {(19843, 22)}
```

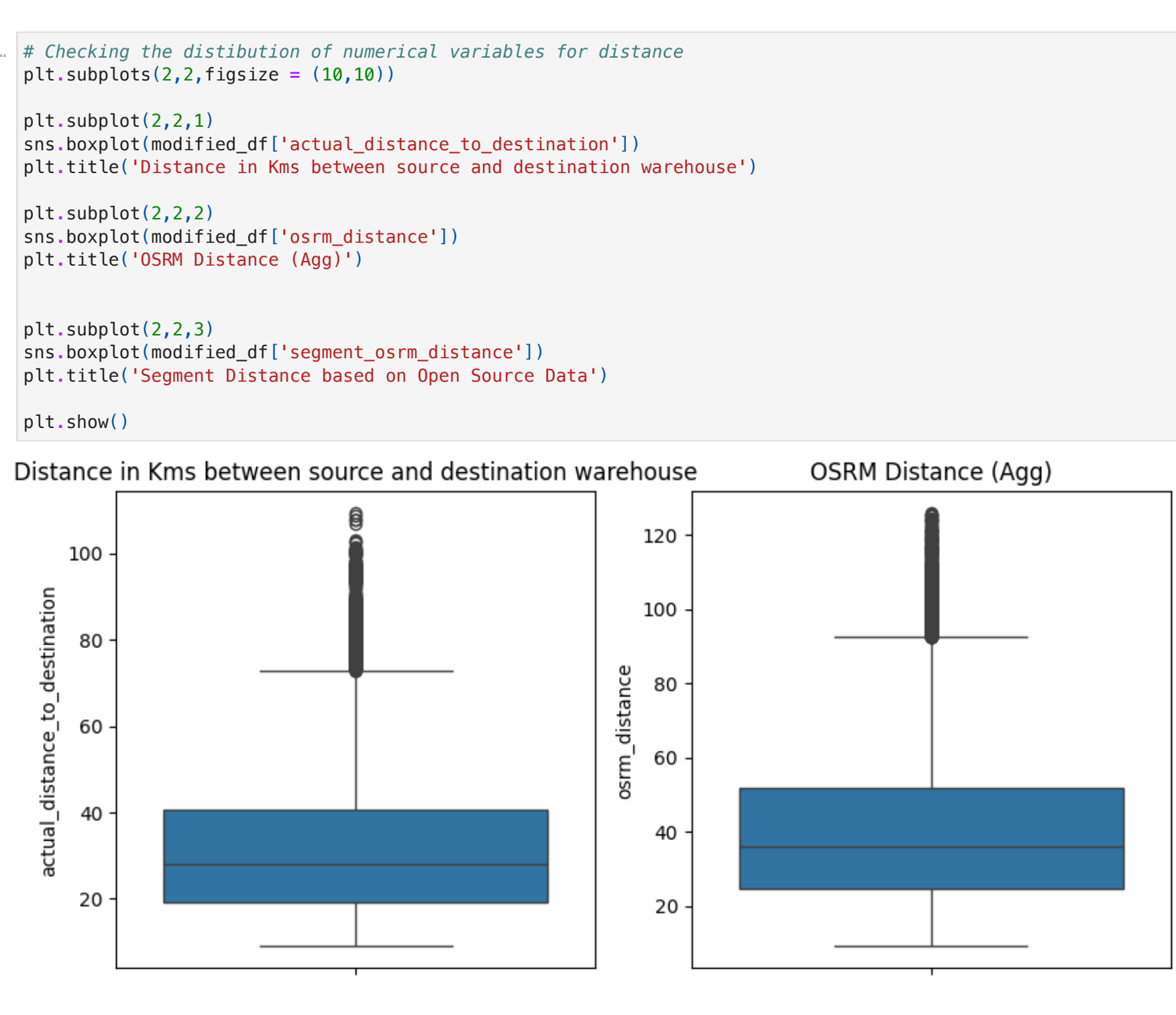
```
# Lets relook at the distrubution of data post removing outliers from duration_source_destiantion column

# Checking the distibution of numerical variables for time
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['duration_source_dest'])
plt.title('Time taken to deliver from source to destination')

plt.subplot(2,2,2)
sns.boxplot(modified_df['actual_time'])
plt.title('Actual Time (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['osrm_time'])
plt.title('Open Source Time (Agg)')


plt.subplot(2,2,4)
sns.boxplot(modified_df['segment_actual_time'])
plt.title('Segment Actual Time)')

plt.show()
```



In [134…
```
# Lets relook at the distrubution of data post removing outliers from duration_source_destiantion column

# Checking the distibution of numerical variables for time
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['duration_source_dest'])
plt.title('Time taken to deliver from source to destination')

plt.subplot(2,2,2)
sns.boxplot(modified_df['actual_time'])
plt.title('Actual Time (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['osrm_time'])
plt.title('Open Source Time (Agg)')


plt.subplot(2,2,4)
sns.boxplot(modified_df['segment_actual_time'])
plt.title('Segment Actual Time)')

plt.show()
```



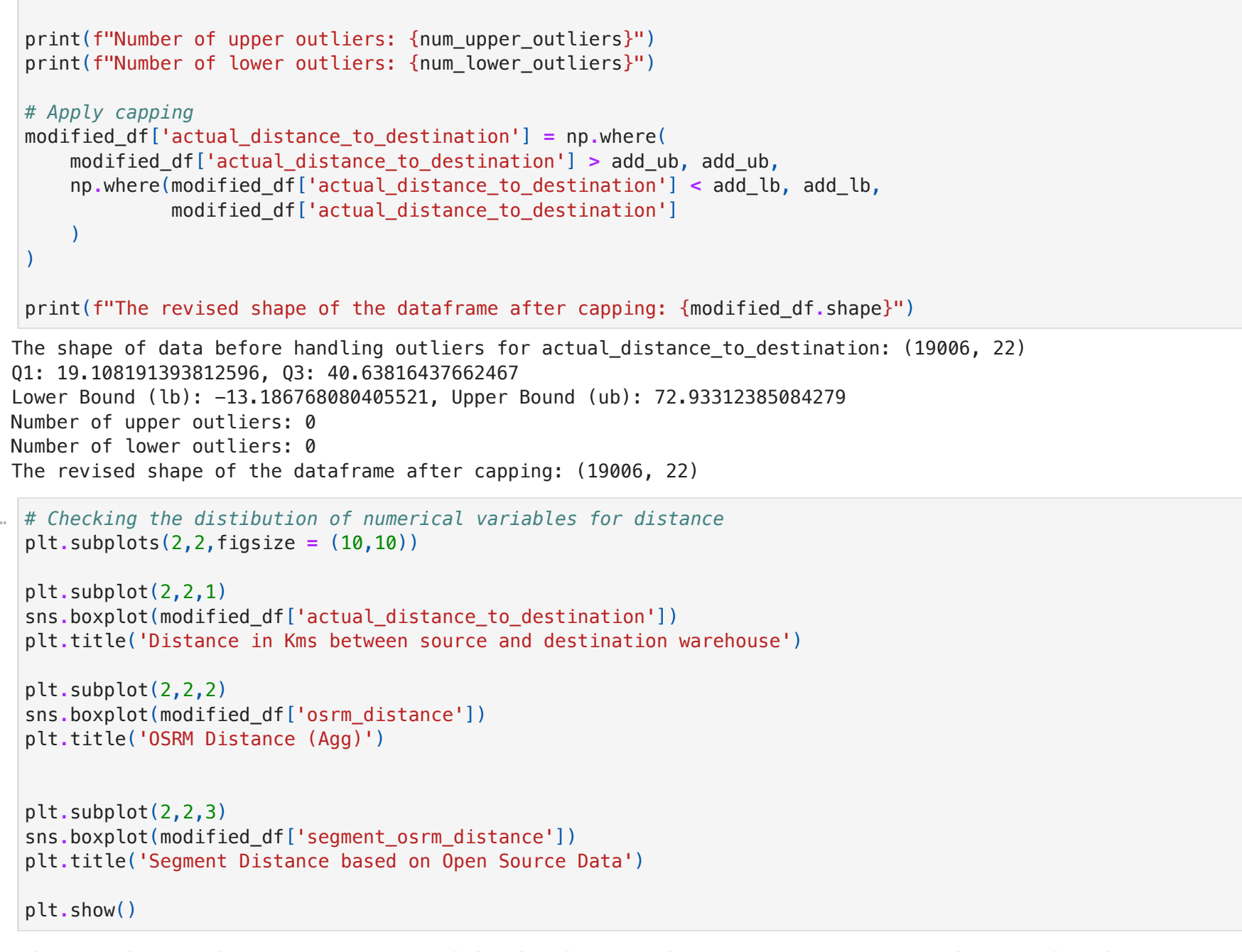
In [135…
```
# Checking the distibution of numerical variables for distance
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['actual_distance_to_destination'])
plt.title('Distance in Kms between source and destination warehouse')

plt.subplot(2,2,2)
sns.boxplot(modified_df['osrm_distance'])
plt.title('OSRM Distance (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['segment_osrm_distance'])
plt.title('Segment Distance based on Open Source Data')

plt.show()
```



In [138…
```
# Treating the outliers on the actual_distance_to_destination using capping

print(f"The shape of data before handling outliers for actual_distance_to_destination: {modified_df.shape}")

add_q1 = np.quantile(modified_df['actual_distance_to_destination'], 0.25)
add_q3 = np.quantile(modified_df['actual_distance_to_destination'], 0.75)

print(f"Q1: {add_q1}, Q3: {add_q3}")

iqr = add_q3 - add_q1

add_ub = add_q3 + 1.5 * iqr
add_lb = add_q1 - 1.5 * iqr

print(f"Lower Bound (lb): {add_lb}, Upper Bound (ub): {add_ub}")

# Count of potential outliers
num_upper_outliers = (modified_df['actual_distance_to_destination'] > add_ub).sum()
num_lower_outliers = (modified_df['actual_distance_to_destination'] < add_lb).sum()

print(f"Number of upper outliers: {num_upper_outliers}")
print(f"Number of lower outliers: {num_lower_outliers}")

# Apply capping
modified_df['actual_distance_to_destination'] = np.where(
    modified_df['actual_distance_to_destination'] > add_ub, add_ub,
    np.where(modified_df['actual_distance_to_destination'] < add_lb, add_lb,
             modified_df['actual_distance_to_destination']
    )
)

print(f"The revised shape of the dataframe after capping: {modified_df.shape}")
```

```
The shape of data before handling outliers for actual_distance_to_destination: (19006, 22)
Q1: 19.108191393812596, Q3: 40.63816437662467
Lower Bound (lb): -13.186768080405521, Upper Bound (ub): 72.93312385084279
Number of upper outliers: 0
Number of lower outliers: 0
The revised shape of the dataframe after capping: (19006, 22)
```

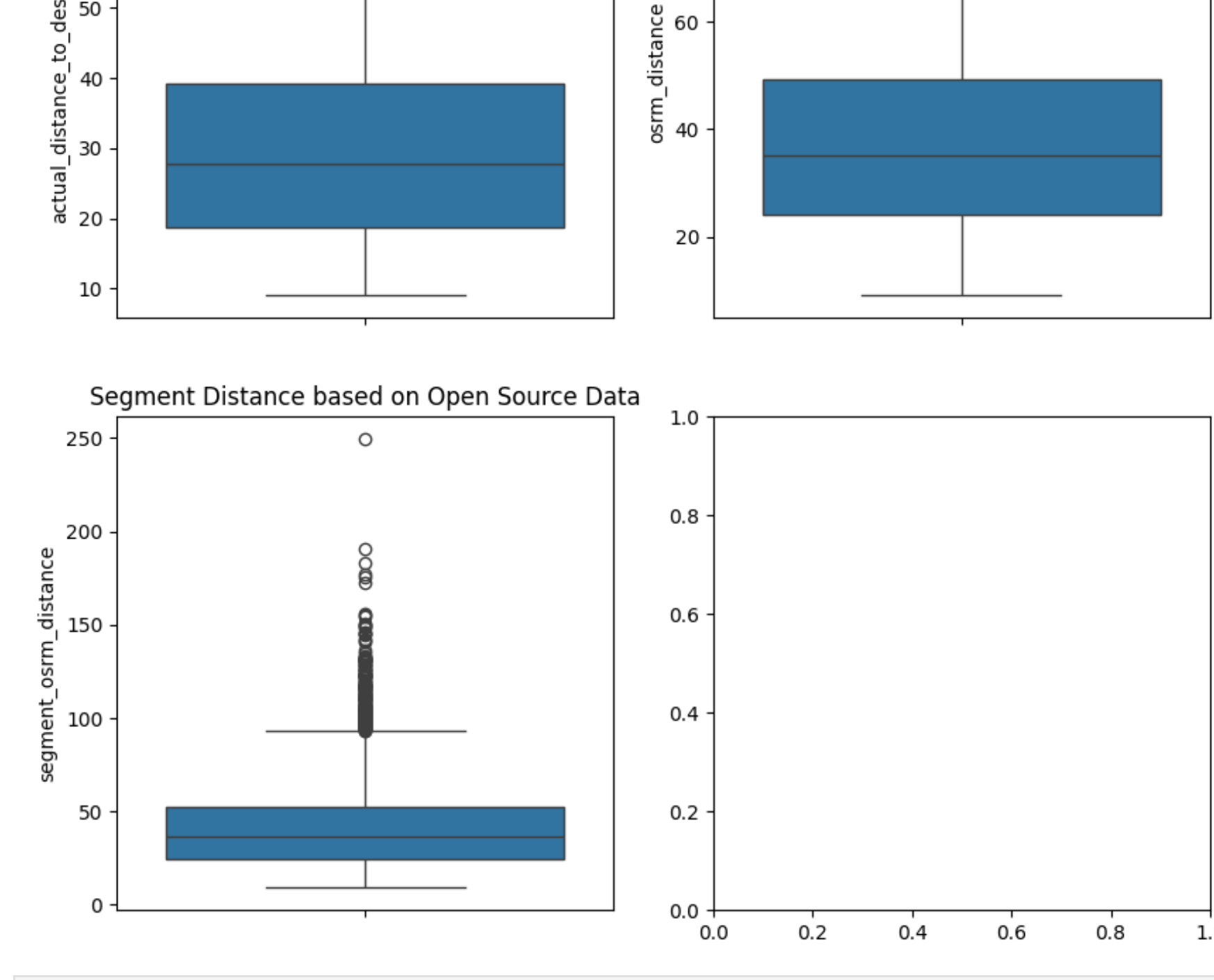
In [139…
```
# Checking the distibution of numerical variables for distance
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['actual_distance_to_destination'])
plt.title('Distance in Kms between source and destination warehouse')

plt.subplot(2,2,2)
sns.boxplot(modified_df['osrm_distance'])
plt.title('OSRM Distance (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['segment_osrm_distance'])
plt.title('Segment Distance based on Open Source Data')

plt.show()
```



In [142…
```
# Treating the outliers on the actual_distance_to_destination using capping

print(f"The shape of data before handling outliers for osrm_distance: {modified_df.shape}")

osrd_q1 = np.quantile(modified_df['osrm_distance'], 0.25)
osrd_q3 = np.quantile(modified_df['osrm_distance'], 0.75)

print(f"Q1: {osrd_q1}, Q3: {osrd_q3}")

iqr = osrd_q3 - osrd_q1

osrd_ub = osrd_q3 + 1.5 * iqr
osrd_lb = osrd_q1 - 1.5 * iqr

print(f"Lower Bound (lb): {osrd_lb}, Upper Bound (ub): {osrd_ub}")

# Count of potential outliers
num_upper_outliers = (modified_df['osrm_distance'] > osrd_ub).sum()
num_lower_outliers = (modified_df['osrm_distance'] < osrd_lb).sum()

print(f"Number of upper outliers: {num_upper_outliers}")
print(f"Number of lower outliers: {num_lower_outliers}")

# Since we have 672 outliers,so we can remove them rather than capping

modified_df = modified_df[(modified_df['osrm_distance']>=osrd_lb) & (modified_df['osrm_distance']<= osrd_ub)]

print(f"The revised shape of the dataframe after removing it: {modified_df.shape}")
```

```
The shape of data before handling outliers for osrm_distance: (19006, 22)
Q1: 24.654925, Q3: 51.768950000000004
Lower Bound (lb): -16.016112500000013, Upper Bound (ub): 92.43998750000002
Number of upper outliers: 672
Number of lower outliers: 0
The revised shape of the dataframe after removing it: (18334, 22)
```

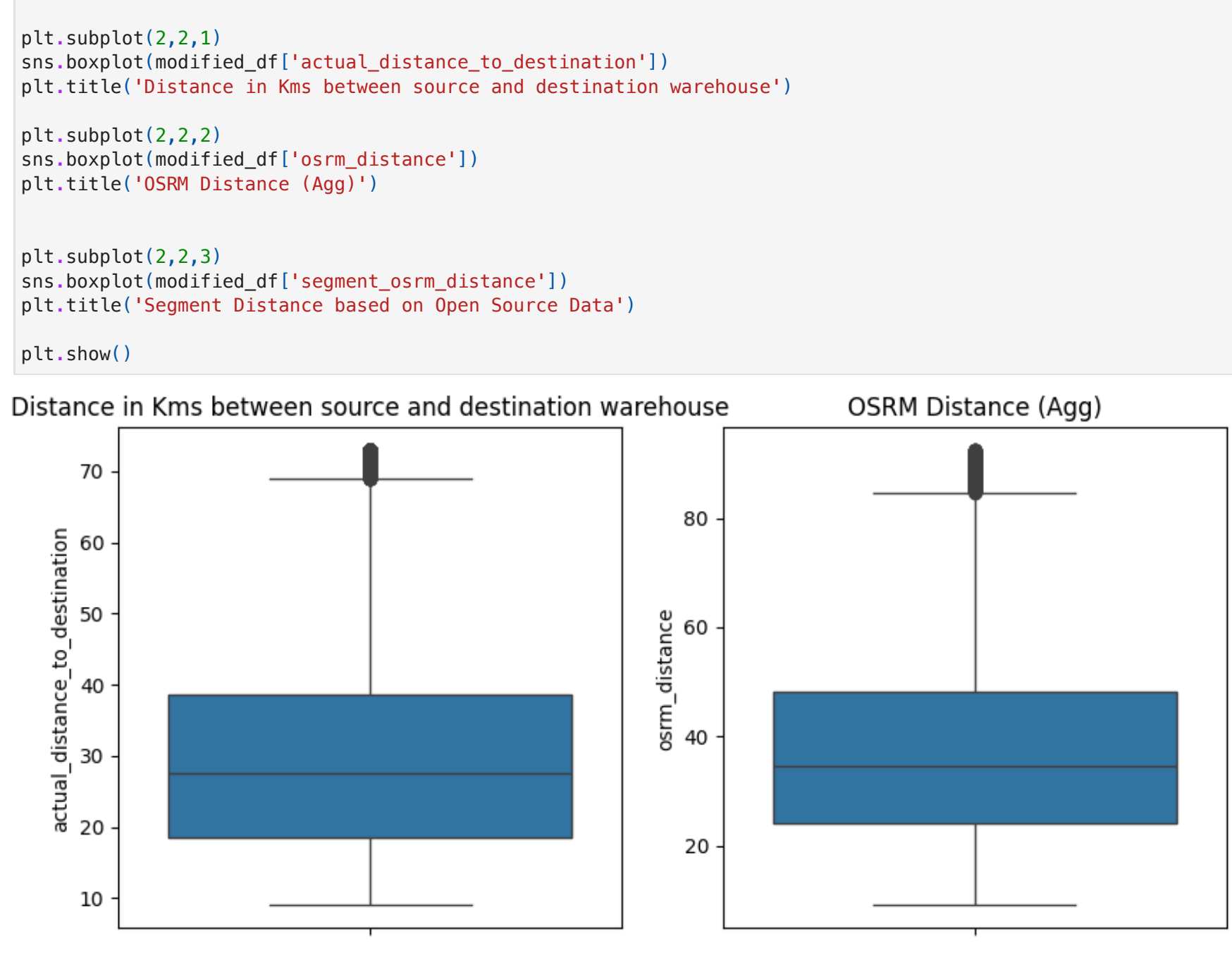
In [143…
```
# Checking the distibution of numerical variables for distance
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['actual_distance_to_destination'])
plt.title('Distance in Kms between source and destination warehouse')

plt.subplot(2,2,2)
sns.boxplot(modified_df['osrm_distance'])
plt.title('OSRM Distance (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['segment_osrm_distance'])
plt.title('Segment Distance based on Open Source Data')

plt.show()
```



In [145…
```
# Treating the outliers on the segment_osrm_distance using capping

print(f"The shape of data before handling outliers for segment_osrm_distance: {modified_df.shape}")

sosd_q1 = np.quantile(modified_df['segment_osrm_distance'], 0.25)
sosd_q3 = np.quantile(modified_df['segment_osrm_distance'], 0.75)

print(f"Q1: {sosd_q1}, Q3: {sosd_q3}")

iqr = sosd_q3 - sosd_q1

sosd_ub = sosd_q3 + 1.5 * iqr
sosd_lb = sosd_q1 - 1.5 * iqr

print(f"Lower Bound (lb): {sosd_lb}, Upper Bound (ub): {sosd_ub}")

# Count of potential outliers
num_upper_outliers = (modified_df['segment_osrm_distance'] > sosd_ub).sum()
num_lower_outliers = (modified_df['segment_osrm_distance'] < sosd_lb).sum()

print(f"Number of upper outliers: {num_upper_outliers}")
print(f"Number of lower outliers: {num_lower_outliers}")

# Since we have 260 outliers,so we can remove them rather than capping

modified_df = modified_df[(modified_df['segment_osrm_distance']>=sosd_lb) & (modified_df['segment_osrm_distance']<= sosd_ub)]


print(f"The revised shape of the dataframe after removing it: {modified_df.shape}")
```

```
The shape of data before handling outliers for segment_osrm_distance: (18334, 22)
Q1: 24.7347, Q3: 52.189475
Lower Bound (lb): -16.447462500000004, Upper Bound (ub): 93.3716375
Number of upper outliers: 260
Number of lower outliers: 0
The revised shape of the dataframe after removing it: (18074, 22)
```

In [146…
```
# Checking the distibution of numerical variables for distance
plt.subplots(2,2,figsize = (10,10))

plt.subplot(2,2,1)
sns.boxplot(modified_df['actual_distance_to_destination'])
plt.title('Distance in Kms between source and destination warehouse')

plt.subplot(2,2,2)
sns.boxplot(modified_df['osrm_distance'])
plt.title('OSRM Distance (Agg)')


plt.subplot(2,2,3)
sns.boxplot(modified_df['segment_osrm_distance'])
plt.title('Segment Distance based on Open Source Data')

plt.show()
```



Handling Categorical Variable

In [ ]:
```
# One-hot encode the 'route_type' column

road_type_dummies = pd.get_dummies(modified_df['route_type']).astype(int)
```

In [75]:
```
# Concatenate the new dummy variables with the original DataFrame

modified_df = pd.concat([modified_df,road_type_dummies],axis = 1)
```

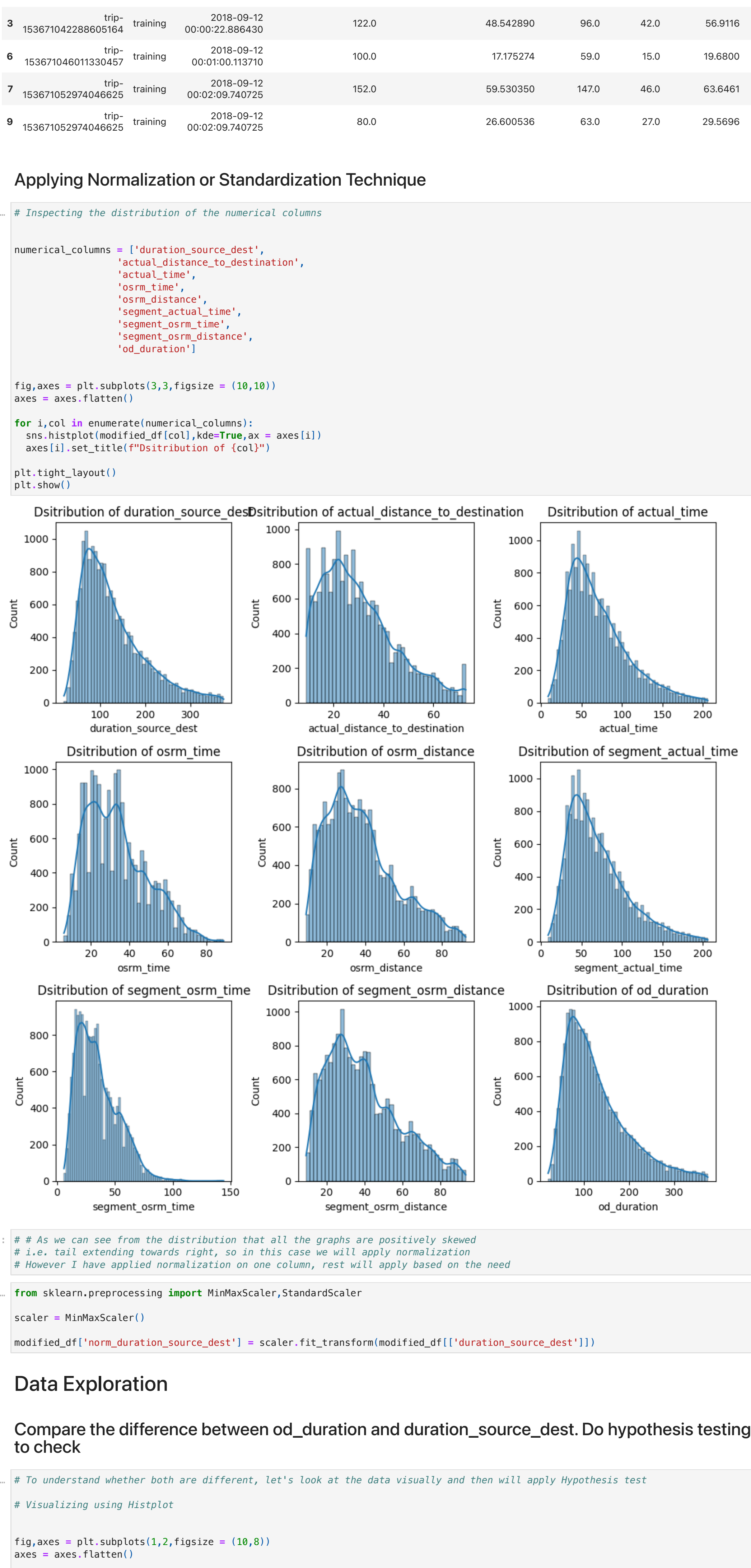
In [76]:
```
# Dropping the route type column

modified_df.drop(columns = ['route_type'],inplace = True)
```

In [148…
```
# Display the updated DataFrame with the new columns

display(modified_df.head())
```

	trip_uuid	data	trip_creation_time	duration_source_dest	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segmen
2	trip-153671042288605164	training	2018-09-12 00:00:22.886430	58.0	24.644021	47.0	26.0	28.1994	46.0	
3	trip-153671042288605164	training	2018-09-12 00:00:22.886430	122.0	48.542890	96.0	42.0	56.9116	95.0	
6	trip-153671046011330457	training	2018-09-12 00:01:00.113710	100.0	17.175274	59.0	15.0	19.6800	59.0	
7	trip-153671052974046625	training	2018-09-12 00:02:09.740725	152.0	59.530350	147.0	46.0	63.6461	147.0	
9	trip-153671052974046625	training	2018-09-12 00:02:09.740725	80.0	26.600536	63.0	27.0	29.5696	63.0	

Applying Normalization or Standardization Technique

In [152…
```
# Inspecting the distribution of the numerical columns


numerical_columns = ['duration_source_dest',
                     'actual_distance_to_destination',
                     'actual_time',
                     'osrm_time',
                     'osrm_distance',
                     'segment_actual_time',
                     'segment_osrm_time',
                     'segment_osrm_distance',
                     'od_duration']


fig,axes = plt.subplots(3,3,figsize = (10,10))
axes = axes.flatten()

for i,col in enumerate(numerical_columns):
  sns.histplot(modified_df[col],kde=True,ax = axes[i])
  axes[i].set_title(f"Dsitribution of {col}")

plt.tight_layout()
plt.show()
```



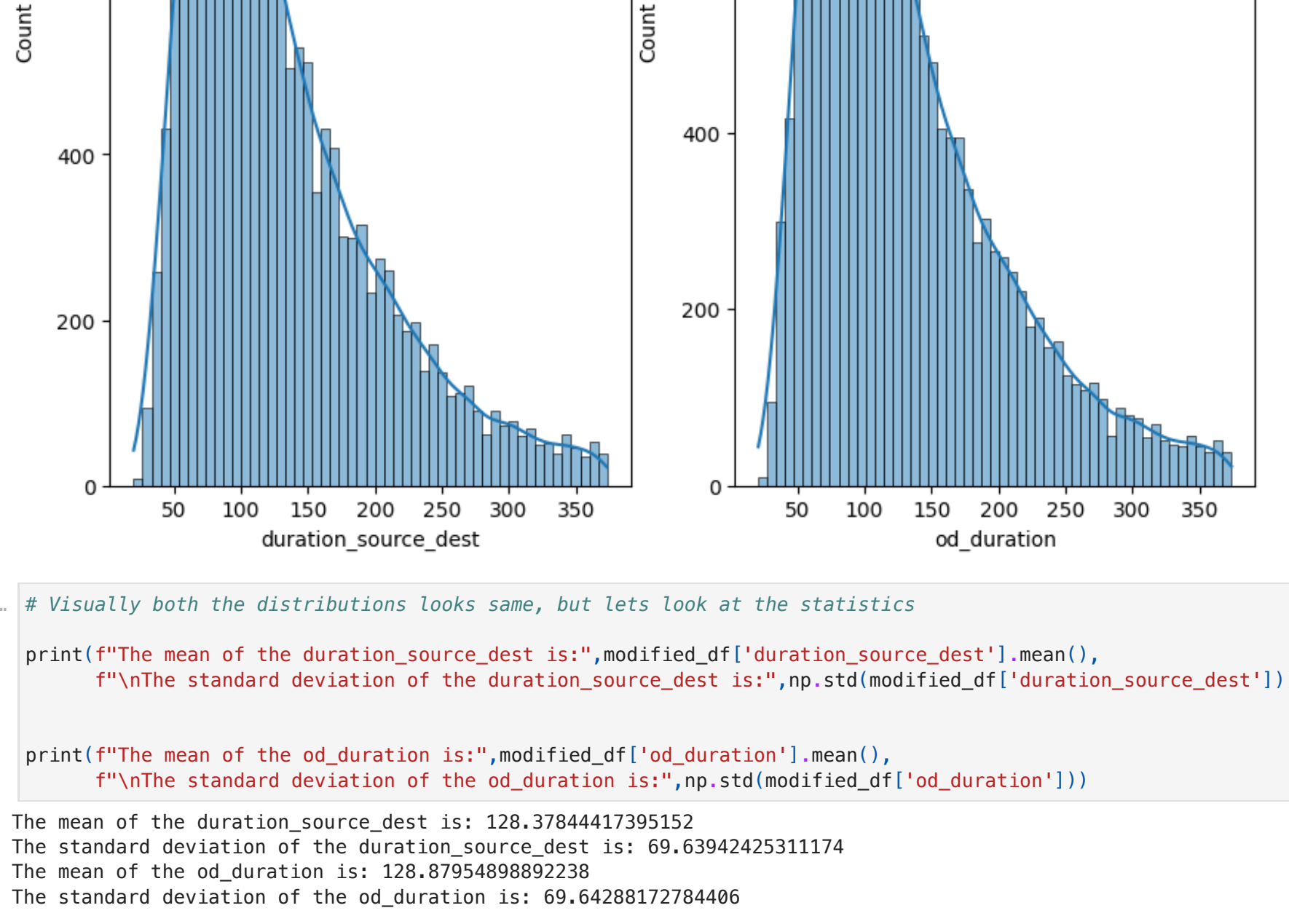
In [ ]:
```
# # As we can see from the distribution that all the graphs are positively skewed
# i.e. tail extending towards right, so in this case we will apply normalization
# However I have applied normalization on one column, rest will apply based on the need
```

In [158…
```
from sklearn.preprocessing import MinMaxScaler,StandardScaler

scaler = MinMaxScaler()

modified_df['norm_duration_source_dest'] = scaler.fit_transform(modified_df[['duration_source_dest']])
```

Data Exploration

Compare the difference between od_duration and duration_source_dest. Do hypothesis testing/ Visual analysis to check

In [160…
```
# To understand whether both are different, let's look at the data visually and then will apply Hypothesis test

# Visualizing using Histplot


fig,axes = plt.subplots(1,2,figsize = (10,8))
axes = axes.flatten()

sns.histplot(modified_df['duration_source_dest'],kde = True,ax = axes[0])
axes[0].set_title('Distribution of duration_source_dest')

sns.histplot(modified_df['od_duration'],kde = True,ax = axes[1])
axes[1].set_title('Distribution of od_duration')

plt.show()
```

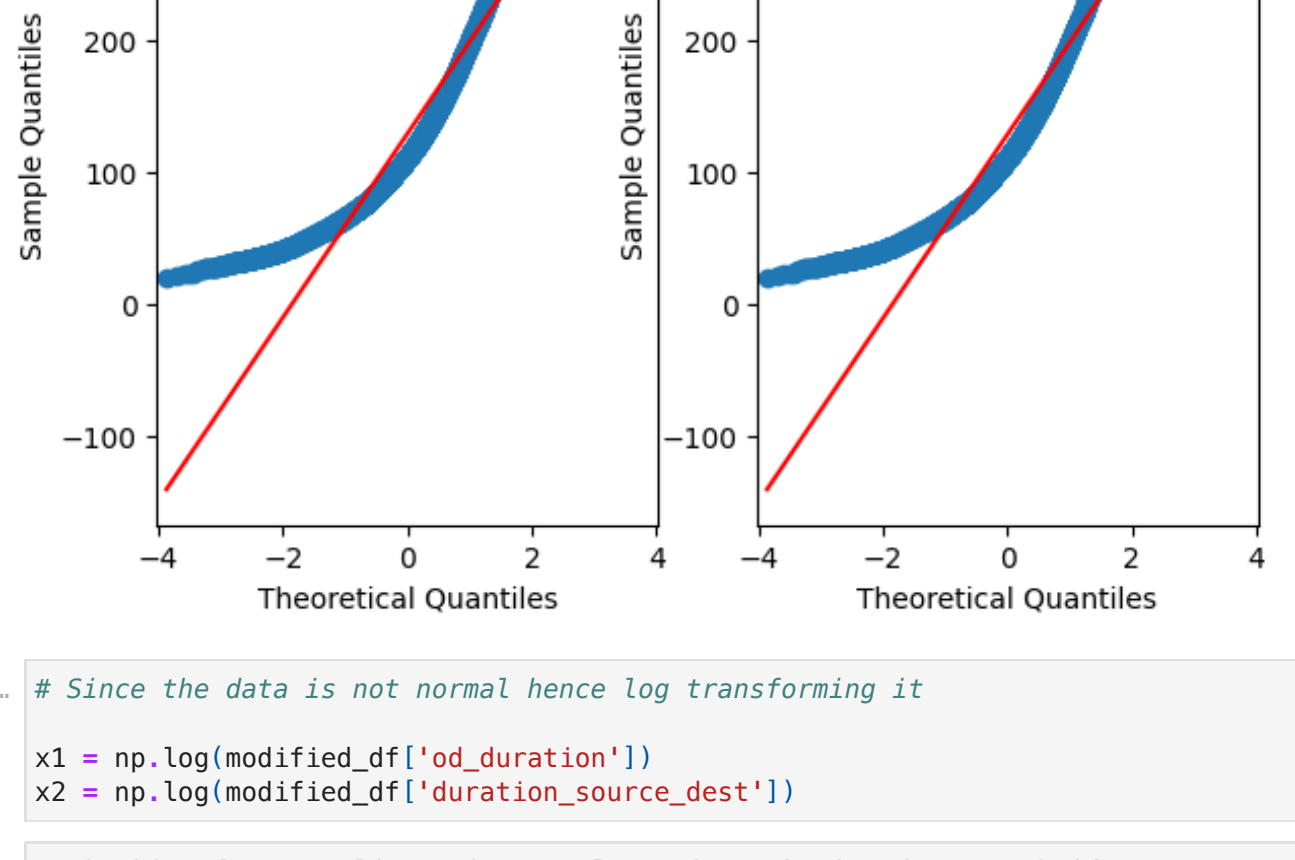


In [162…
```
# Visually both the distributions looks same, but lets look at the statistics

print(f"The mean of the duration_source_dest is:",modified_df['duration_source_dest'].mean(),
      f"\nThe standard deviation of the duration_source_dest is:",np.std(modified_df['duration_source_dest']))


print(f"The mean of the od_duration is:",modified_df['od_duration'].mean(),
      f"\nThe standard deviation of the od_duration is:",np.std(modified_df['od_duration']))
```

```
The mean of the duration_source_dest is: 128.37844417395152
The standard deviation of the duration_source_dest is: 69.63942425311174
The mean of the od_duration is: 128.87954898892238
The standard deviation of the od_duration is: 69.64288172784406
```

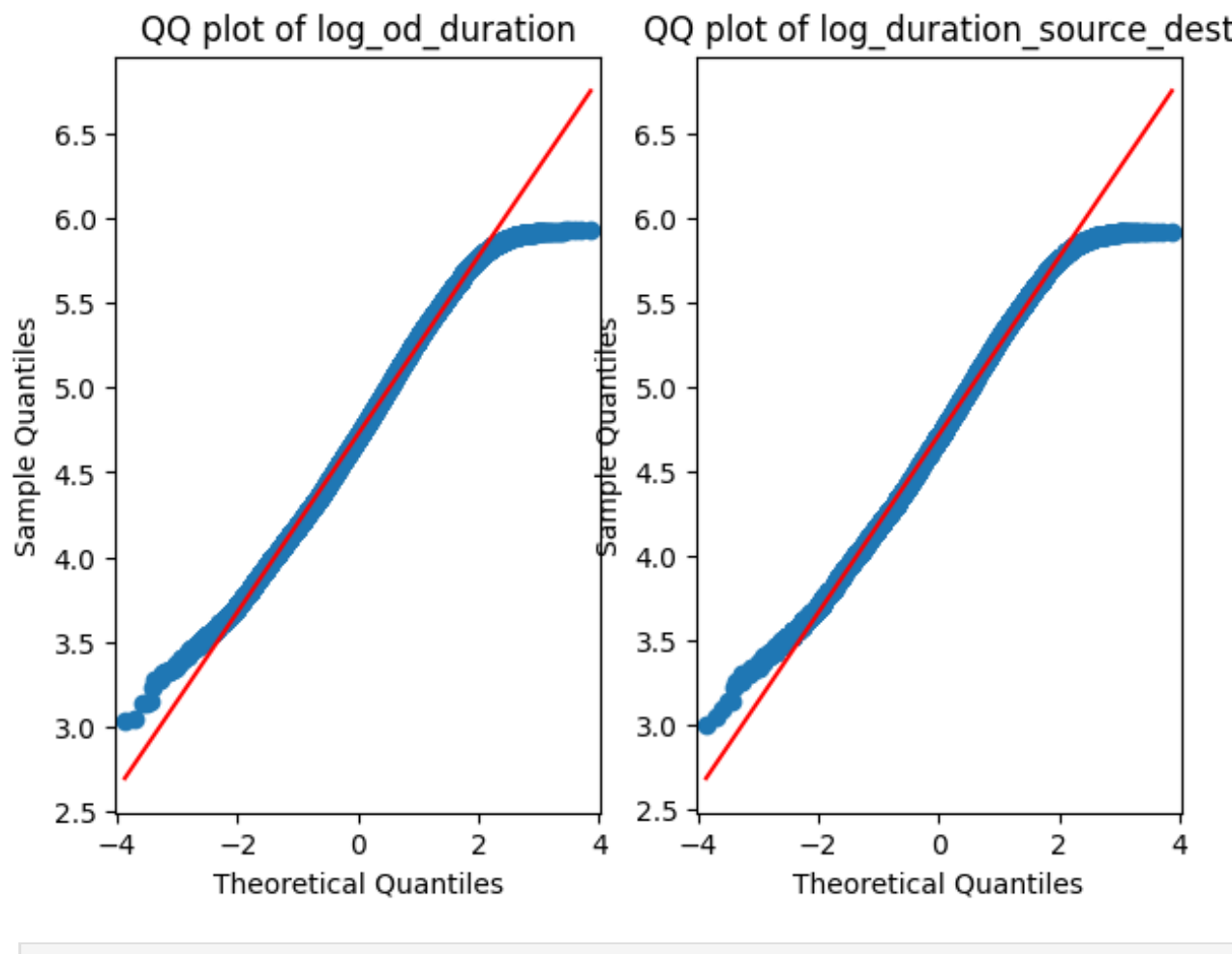
In [ ]:
```
# Both visually and stasticial summary also says that both are same and there is no difference, however lets do hypothesis testing
```

In [171…
```
# Since the sample size is more than 30, and assuming it follows normal distribution I would be using ttest dependent

H0 = "There is no difference between od_duration and duration_source_dest"
H1 = "There is a difference between od_duration and duration_source_dest"

# Checking for normality using QQ plot, since Shapiro doest work if n>200

import statsmodels.api as sm

fig,axes = plt.subplots(1,2,figsize=(7,5))
axes = axes.flatten()

sm.qqplot(modified_df['od_duration'],line = 's',ax=axes[0])
axes[0].set_title("QQ plot of od_duration")

sm.qqplot(modified_df['od_duration'],line = 's',ax=axes[1])
axes[1].set_title("QQ plot of duration_source_dest")

plt.show()
```



In [170…
```
# Since the data is not normal hence log transforming it

x1 = np.log(modified_df['od_duration'])
x2 = np.log(modified_df['duration_source_dest'])
```

In [172…
```
# Checking for normality using QQ plot, since Shapiro doest work if n>200

# import statsmodels.api as sm

fig,axes = plt.subplots(1,2,figsize=(7,5))
axes = axes.flatten()

sm.qqplot(x1,line = 's',ax=axes[0])
axes[0].set_title("QQ plot of log_od_duration")

sm.qqplot(x2,line = 's',ax=axes[1])
axes[1].set_title("QQ plot of log_duration_source_dest")

plt.show()
```



In [174…
```
# I would be performing ttest independent test
```

```
Ho = "There is no difference between od_duration and duration_source_dest"
H1 = "There is a significant difference between od_duration and duration_source_dest"

statistic, p_value = ttest_ind(x1,x2)

print(f"T-test Statistic: {statistic:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05  # Significance level

if p_value < alpha:
    print("Reject the null hypothesis")
    print(H1)
else:
    print("Fail to reject the null hypothesis")
    print(H0)
```

```
T-test Statistic: 0.9252
P-value: 0.3549
Fail to reject the null hypothesis
There is no difference between od_duration and duration_source_dest
```

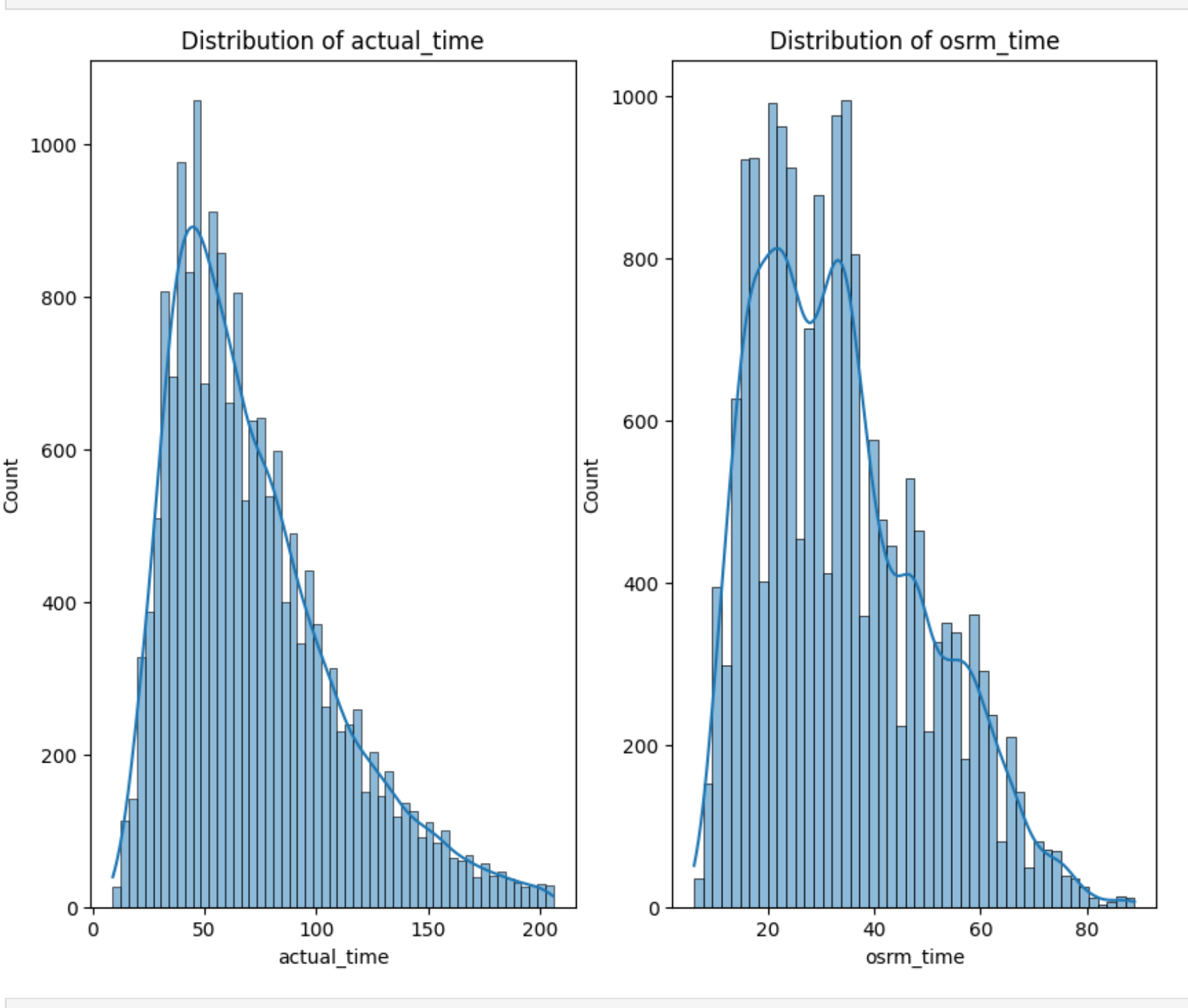
In [ ]:

```
# Hypothesis testing its proved that there is no difference between the two
```

Do hypothesis testing/ visual analysis between actual_time aggregated value and OSRM time aggregated value

In [175…

```
# To understand whether both are different, let's look at the data visually and then will apply Hypothesis test

# Visualizing using Histplot


fig,axes = plt.subplots(1,2,figsize = (10,8))
axes = axes.flatten()

sns.histplot(modified_df['actual_time'],kde = True,ax = axes[0])
axes[0].set_title('Distribution of actual_time')

sns.histplot(modified_df['osrm_time'],kde = True,ax = axes[1])
axes[1].set_title('Distribution of osrm_time')

plt.show()
```



In [176…

```
# Visually the distributions looks different, but lets look at the statistics summary


modified_df[['actual_time','osrm_time']].describe()
```

Out[176…

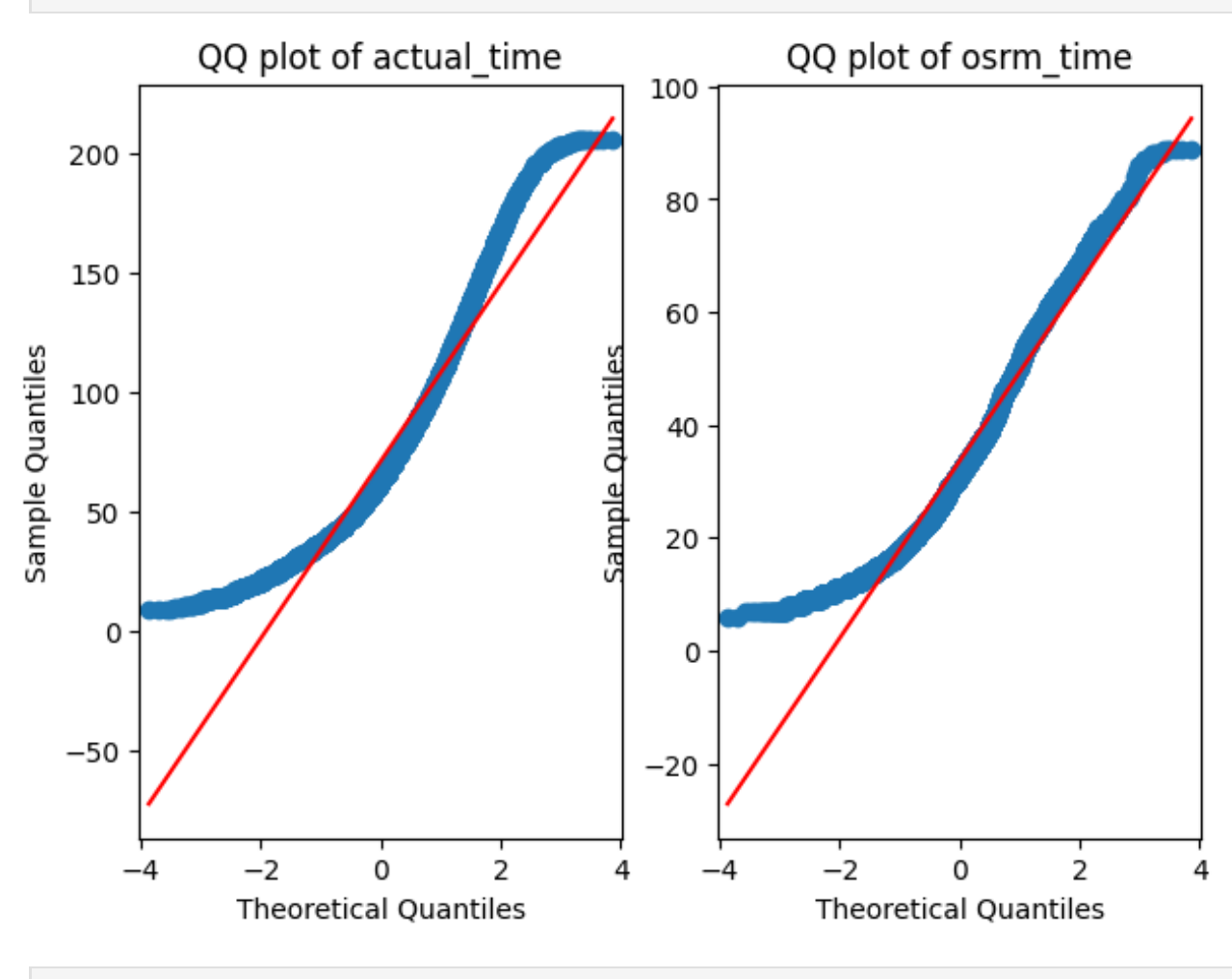
	actual_time	osrm_time
count	18074.000000	18074.000000
mean	71.253901	33.570931
std	37.068145	15.713657
min	9.000000	6.000000
25%	44.000000	21.000000
50%	63.000000	31.000000
75%	91.000000	44.000000
max	206.000000	89.000000

In [ ]:

```
# We can see from the above that:

# - Mean of actual_time is greater than osrm_time, which indicates that although osrm time is a theoritical calculation
#   however it is not accurate
# - Standard deviation of actual_time is greater than osrm_time
# - Median of actual_time is greater than osrm_time
```

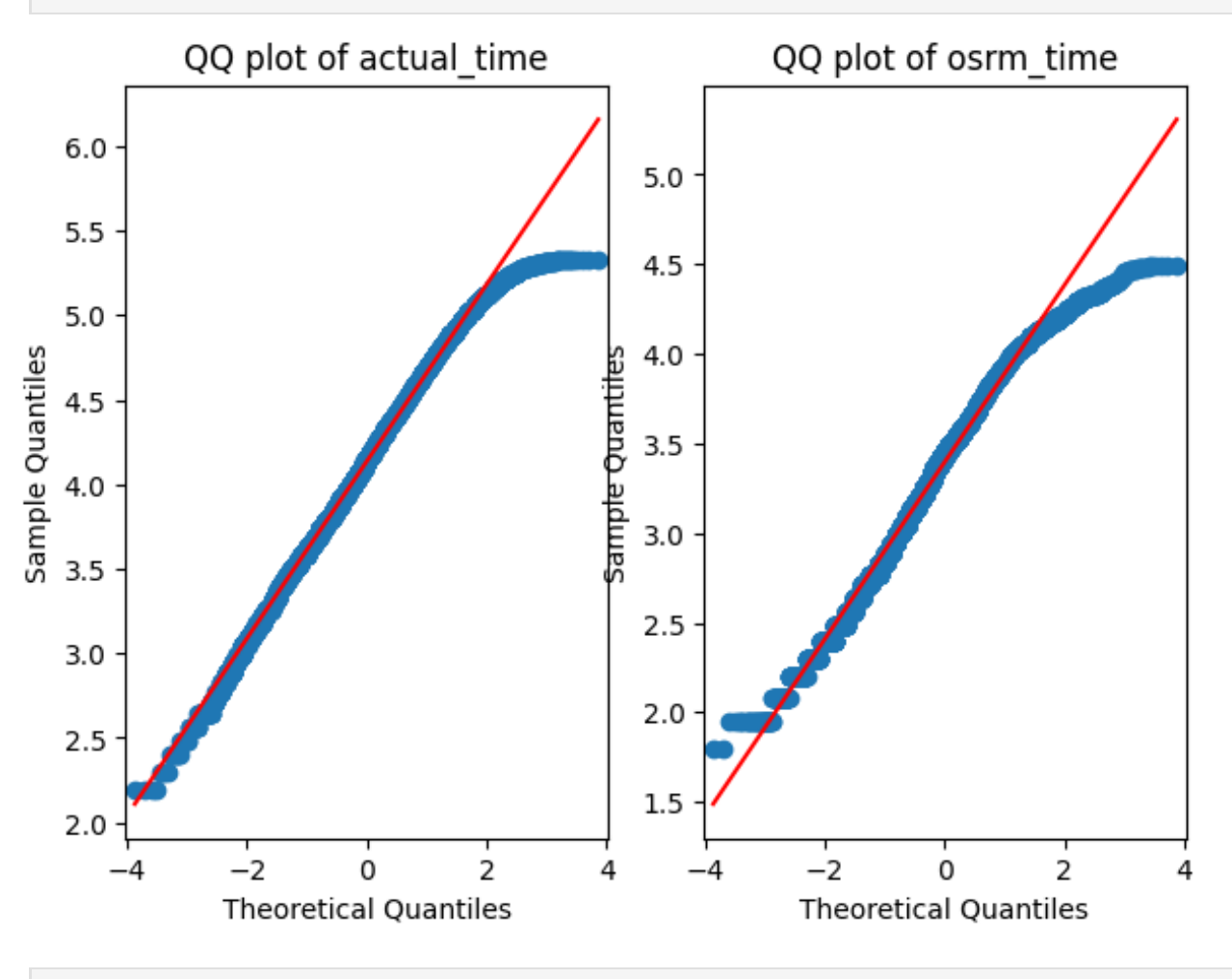
In [177…

```
# Since the sample size is more than 30, and assuming it follows normal distribution I would be using ttest dependent

# Checking for normality using QQ plot, since Shapiro doest work if n>200

import statsmodels.api as sm

fig,axes = plt.subplots(1,2,figsize=(7,5))
axes = axes.flatten()

sm.qqplot(modified_df['actual_time'],line = 's',ax=axes[0])
axes[0].set_title("QQ plot of actual_time")

sm.qqplot(modified_df['osrm_time'],line = 's',ax=axes[1])
axes[1].set_title("QQ plot of osrm_time")

plt.show()
```



In [178…

```
# OSRM time is following normal distribution however thats not the cae of actual time, but using logarithmic transformation for
# both columns

x3 = np.log(modified_df['actual_time'])
x4 = np.log(modified_df['osrm_time'])


fig,axes = plt.subplots(1,2,figsize=(7,5))
axes = axes.flatten()

sm.qqplot(x3,line = 's',ax=axes[0])
axes[0].set_title("QQ plot of actual_time")

sm.qqplot(x4,line = 's',ax=axes[1])
axes[1].set_title("QQ plot of osrm_time")

plt.show()
```



In [179…

```
# I would be performing ttest independent test
Ho = "There is no difference between actual_time and osrm_time"
H1 = "There is a significant difference between actual_time and osrm_time"

statistic, p_value = ttest_ind(x3,x4)

print(f"T-test Statistic: {statistic:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05  # Significance level

if p_value < alpha:
    print("Reject the null hypothesis")
    print(H1)
else:
    print("Fail to reject the null hypothesis")
    print(H0)
```

```
T-test Statistic: 137.3475
P-value: 0.0000
Reject the null hypothesis
There is a significant difference between actual_time and osrm_time
```

In [180…

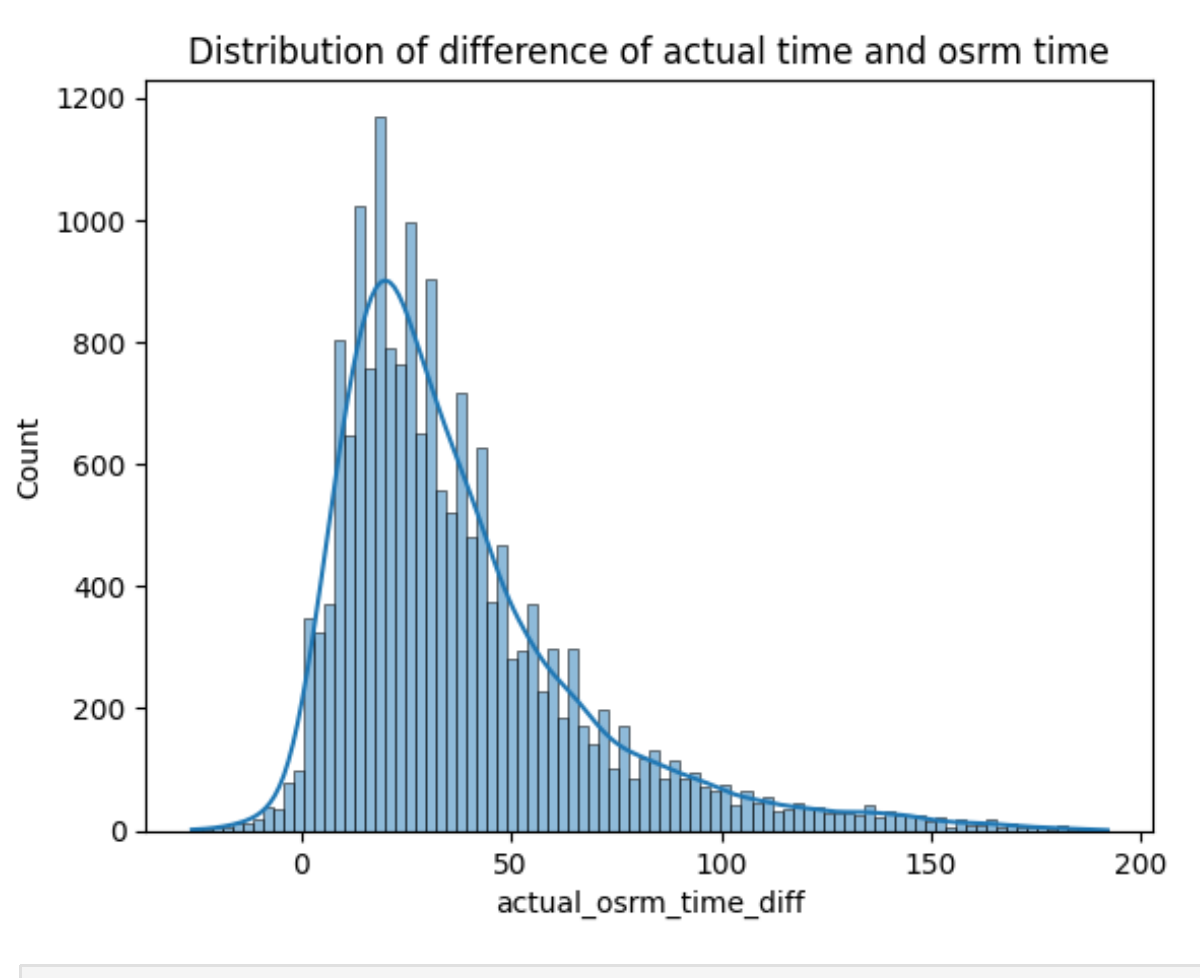
```
# Hypothesis testing also suggests there is a difference, however it would be interesting to see the quantum of difference and analyzing it
# with other variables

modified_df['actual_osrm_time_diff'] = modified_df['actual_time'] - modified_df['osrm_time']
```

In [181…

```
# Lets draw the hist plot along with KDE to understand the distribution

sns.histplot(modified_df['actual_osrm_time_diff'],kde = True)
plt.title('Distribution of difference of actual time and osrm time')
```

Out[181…

```
Text(0.5, 1.0, 'Distribution of difference of actual time and osrm time')
```



In [182…

```
modified_df['actual_osrm_time_diff'].describe()
```

Out[182…

	actual_osrm_time_diff
count	18074.000000
mean	37.682970
std	30.010737
min	-26.000000
25%	17.000000
50%	30.000000
75%	49.000000
max	192.000000

dtype: float64

In [184…

```
# - The above data shows that on an average there is 37 mins difference between actual time and osrm time with a standard deviation of
#   30 mins.
# - And in same cases the difference is -ve which means there are some trips with actual time less than theoritical time.
```

In [185…

```
modified_df.head()
```

Out[185…

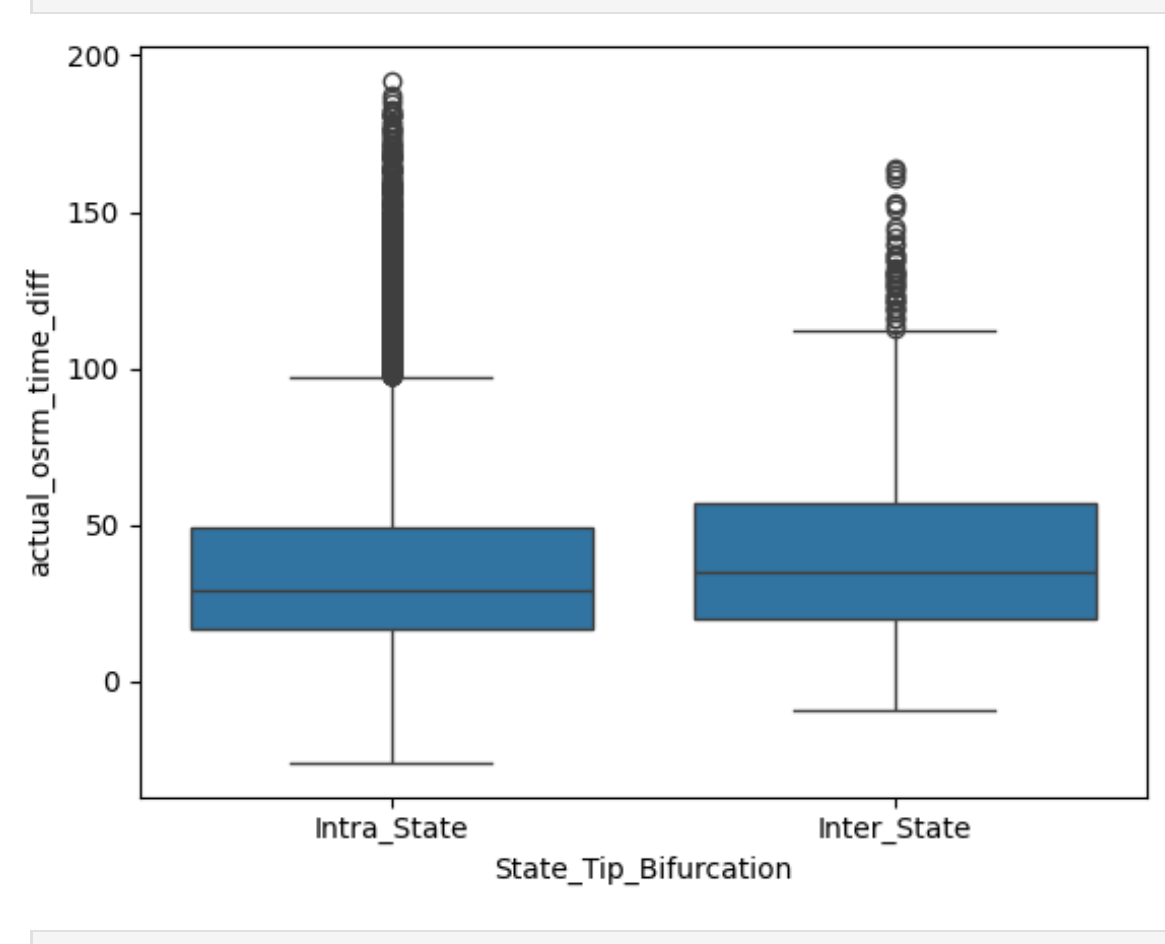
	trip_uuid	data	trip_creation_time	duration_source_dest	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segm
2	trip-153671042288605164	training	2018-09-12 00:00:22.886430	58.0	24.644021	47.0	26.0	28.1994	46.0	
3	trip-153671042288605164	training	2018-09-12 00:00:22.886430	122.0	48.542890	96.0	42.0	56.9116	95.0	
6	trip-153671046011330457	training	2018-09-12 00:01:00.113710	100.0	17.175274	59.0	15.0	19.6800	59.0	
7	trip-153671052974046625	training	2018-09-12 00:02:09.740725	152.0	59.530350	147.0	46.0	63.6461	147.0	
9	trip-153671052974046625	training	2018-09-12 00:02:09.740725	80.0	26.600536	63.0	27.0	29.5696	63.0	

In [186…

```
modified_df['State_Tip_Bifurcation'] = np.where(modified_df['source_state']==modified_df['destination_state'],"Intra_State","Inter_State")
```

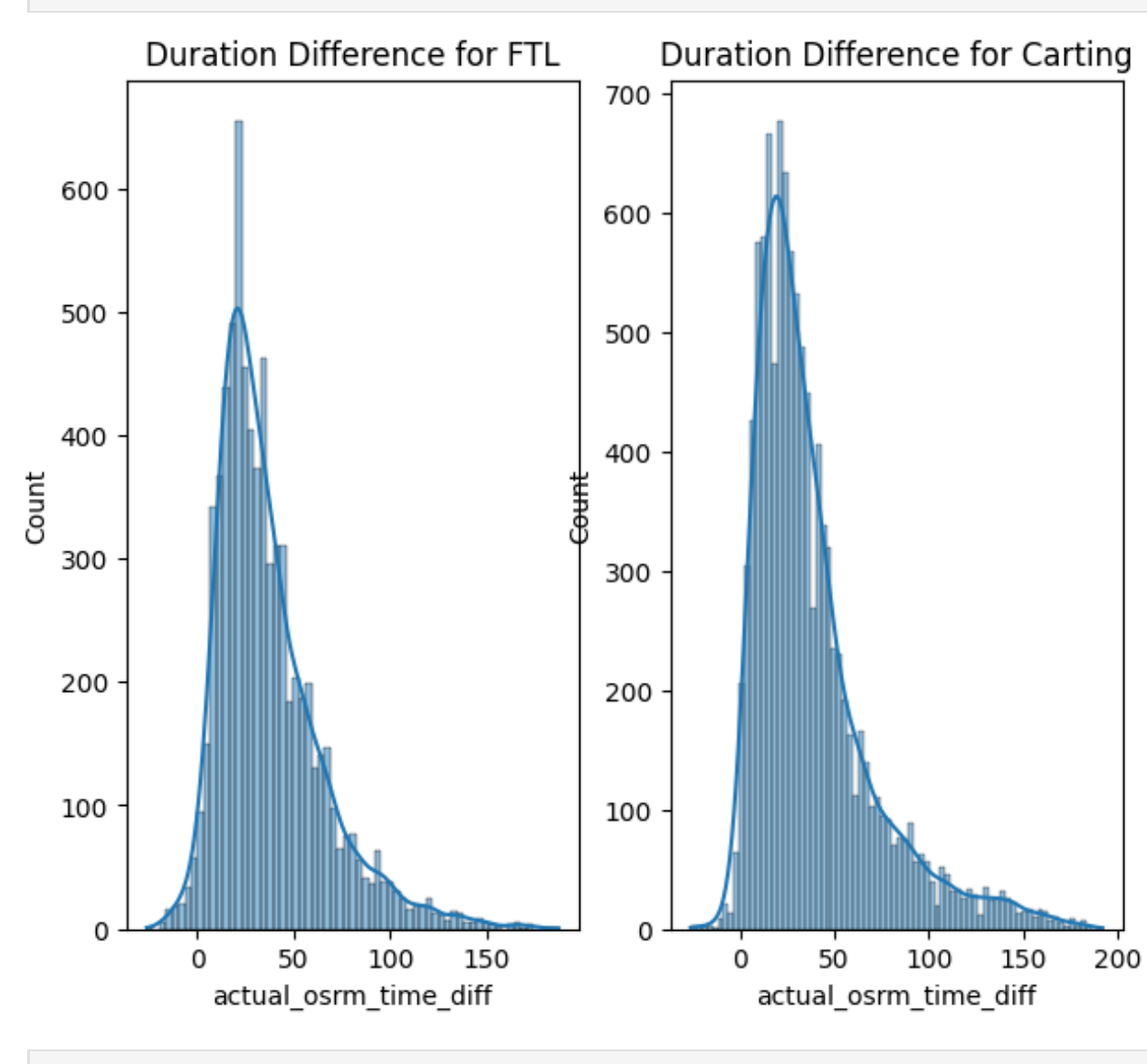
In [189…

```
# Lets understand the difference between actual and osrm time is within the state or outside the state

sns.boxplot(data = modified_df,x = 'State_Tip_Bifurcation',y = 'actual_osrm_time_diff')
plt.show()
```



In [202…

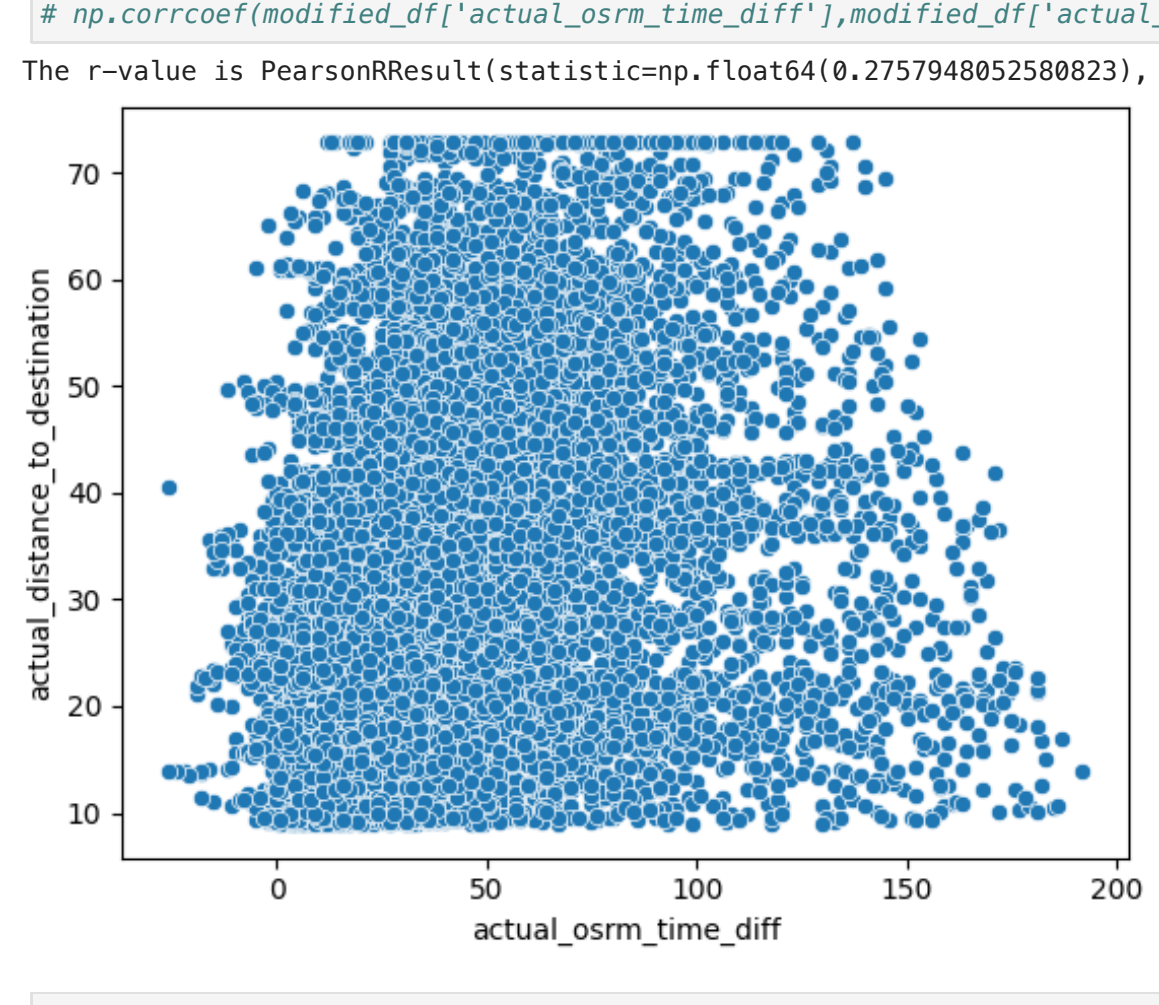
```
fig,axes = plt.subplots(1,2,figsize = (7,6))
aces = axes.flatten()

is_ftl = modified_df[modified_df['FTL'] == 1]
sns.histplot(is_ftl['actual_osrm_time_diff'],kde = True,ax=axes[0])
axes[0].set_title('Duration Difference for FTL')

is_carting = modified_df[modified_df['Carting'] == 1]

sns.histplot(is_carting['actual_osrm_time_diff'],kde = True,ax=axes[1])
axes[1].set_title('Duration Difference for Carting')

plt.show()
```



In [203…

```
# wante to check whether the difference is more in case the distance is increase, i.e. is there any relation, but based on the below
  # graph its clear that there is no correlation between the distance and the difference (actual vs osrm time)


sns.scatterplot(data=modified_df,x='actual_osrm_time_diff',y='actual_distance_to_destination')

# np.corrcoef(modified_df['actual_osrm_time_diff'],modified_df['actual_distance_to_destination'])
```

```
The r-value is PearsonRResult(statistic=np.float64(0.2757948052580823), pvalue=np.float64(7.54864039513e-313))
```



In [205…

```
# Lets look at the from the value perspective

from scipy.stats import pearsonr, spearmanr

print(f"The coefficient value is",pearsonr(modified_df['actual_osrm_time_diff'],modified_df['actual_distance_to_destination']))
```

```
The coefficient value is PearsonRResult(statistic=np.float64(0.2757948052580823), pvalue=np.float64(7.54864039513e-313))
```

In [ ]:

```
# Inferences:

# - OSRM Underestimation is Widespread: OSRM time estimates consistently underestimate actual delivery times, indicating a significant
#   gap between theoretical planning and real-world execution. This impacts forecasting accuracy and customer expectations.
# - Route Type Drives Discrepancy: The type of route (FTL vs. Carting) is a major factor in predicting this time gap.
#   Carting operations show greater and more variable delays, suggesting OSRM models struggle with multi-stop complexities.
# - Distance is Not the Primary Culprit: Surprisingly, the actual distance traveled has only a weak correlation with the time
#   discrepancy. This means focusing solely on optimizing mileage won't solve the core problem of unpredictable delivery times.
# - Actionable Next Steps: Leaders should investigate non-distance factors (e.g., specific geographies, traffic patterns,
#   operational bottlenecks at facilities, time of day, inter-state complexities) to improve predictability, especially for
#   multi-stop and inter-state operations, and consider tailored forecasting models for different route types.
```

Do hypothesis testing/ visual analysis between actual_time aggregated value and segment actual time aggregated value

In [206…

```
# In this we are comparing below two features:

modified_df[['actual_time','segment_actual_time']]
```

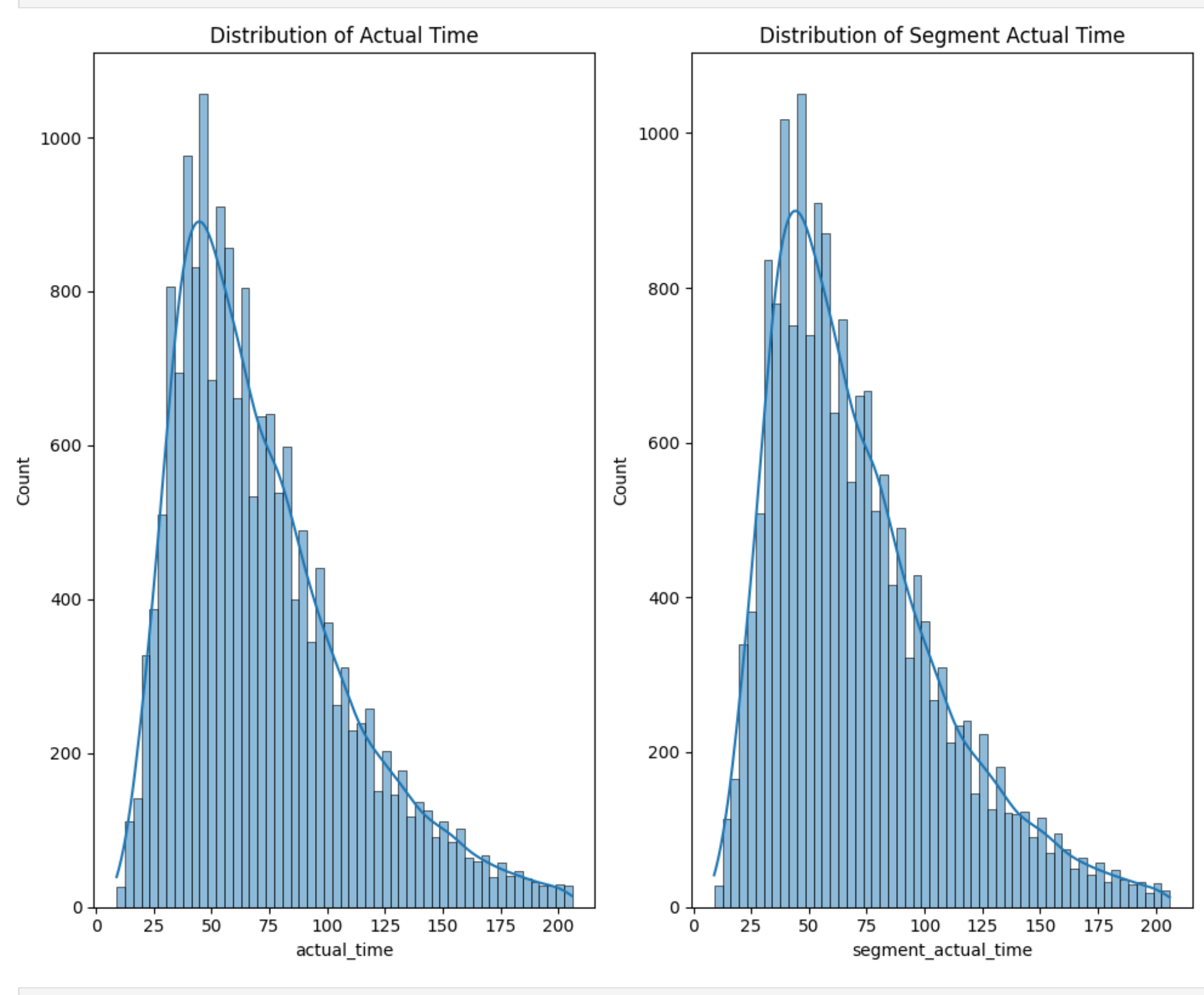
Out[206…

	actual_time	segment_actual_time
2	47.0	46.0
3	96.0	95.0
6	59.0	59.0
7	147.0	147.0
9	63.0	63.0
...	...	...
26362	33.0	32.0
26363	51.0	49.0
26364	90.0	89.0
26365	30.0	29.0
26367	42.0	41.0

18074 rows × 2 columns

In [214…

```
# Lets check vidually:

fig,axes = plt.subplots(1,2,figsize = (10,8))
axes = axes.flatten()

sns.histplot(modified_df['actual_time'],kde = True,ax = axes[0])
axes[0].set_title('Distribution of Actual Time')

sns.histplot(modified_df['segment_actual_time'],kde = True,ax = axes[1])
axes[1].set_title('Distribution of Segment Actual Time')

plt.tight_layout()
plt.show()
```



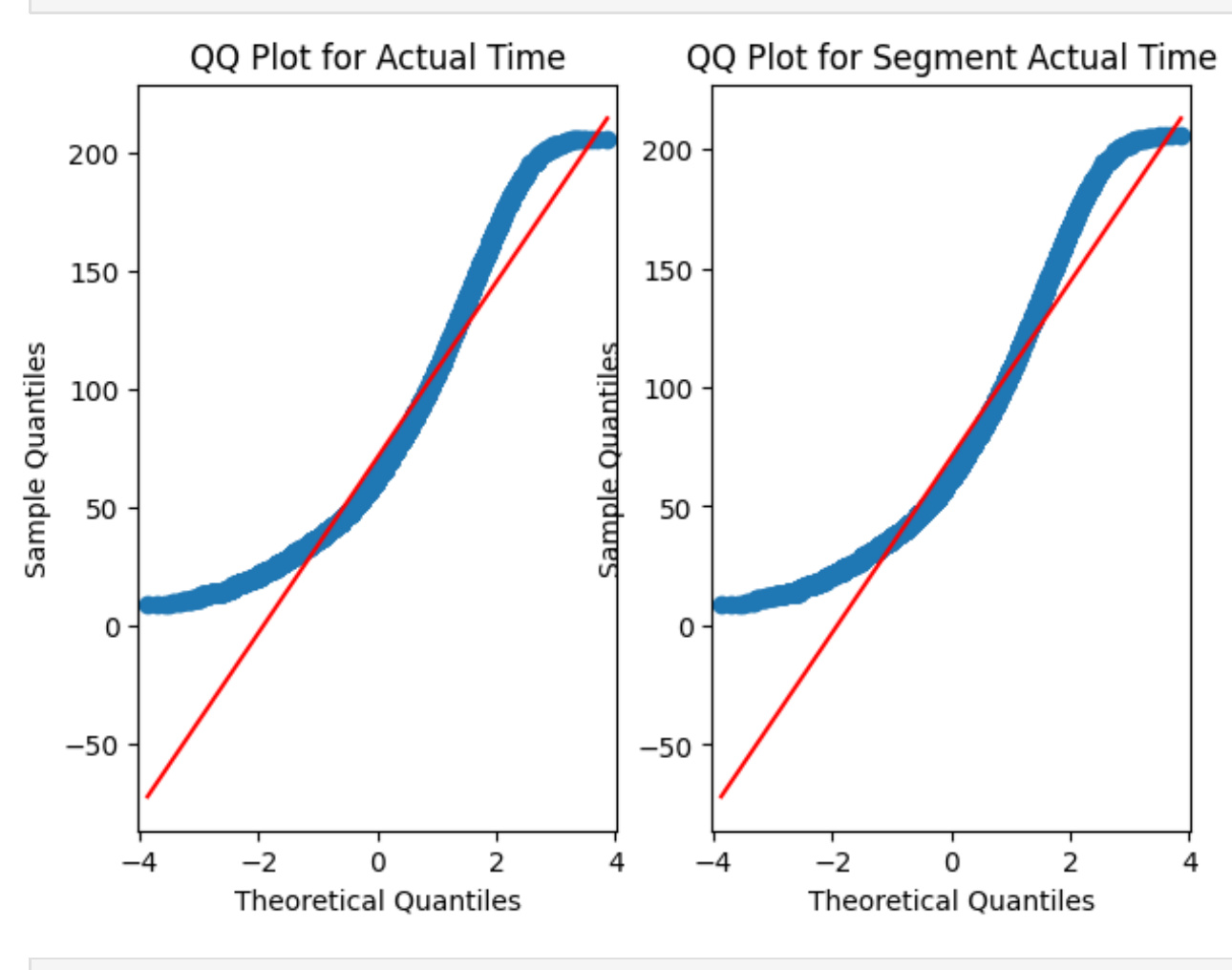
In [215…

```
# Vidually it looks similar, however lets do the statiscial summary and hypothesis testing

print('Descriptive Statistics for actual_time:')
print(modified_df['actual_time'].describe())
print('\nDescriptive Statistics for segment_actual_time:')
print(modified_df['segment_actual_time'].describe())
```

```
Descriptive Statistics for actual_time:
count    18074.000000
mean        71.253901
std         37.068145
min          9.000000
25%         44.000000
50%         63.000000
75%         91.000000
max        206.000000
Name: actual_time, dtype: float64

Descriptive Statistics for segment_actual_time:
count    18074.000000
mean        70.545646
std         36.827934
min          9.000000
25%         43.000000
50%         62.000000
75%         90.000000
max        206.000000
Name: segment_actual_time, dtype: float64
```

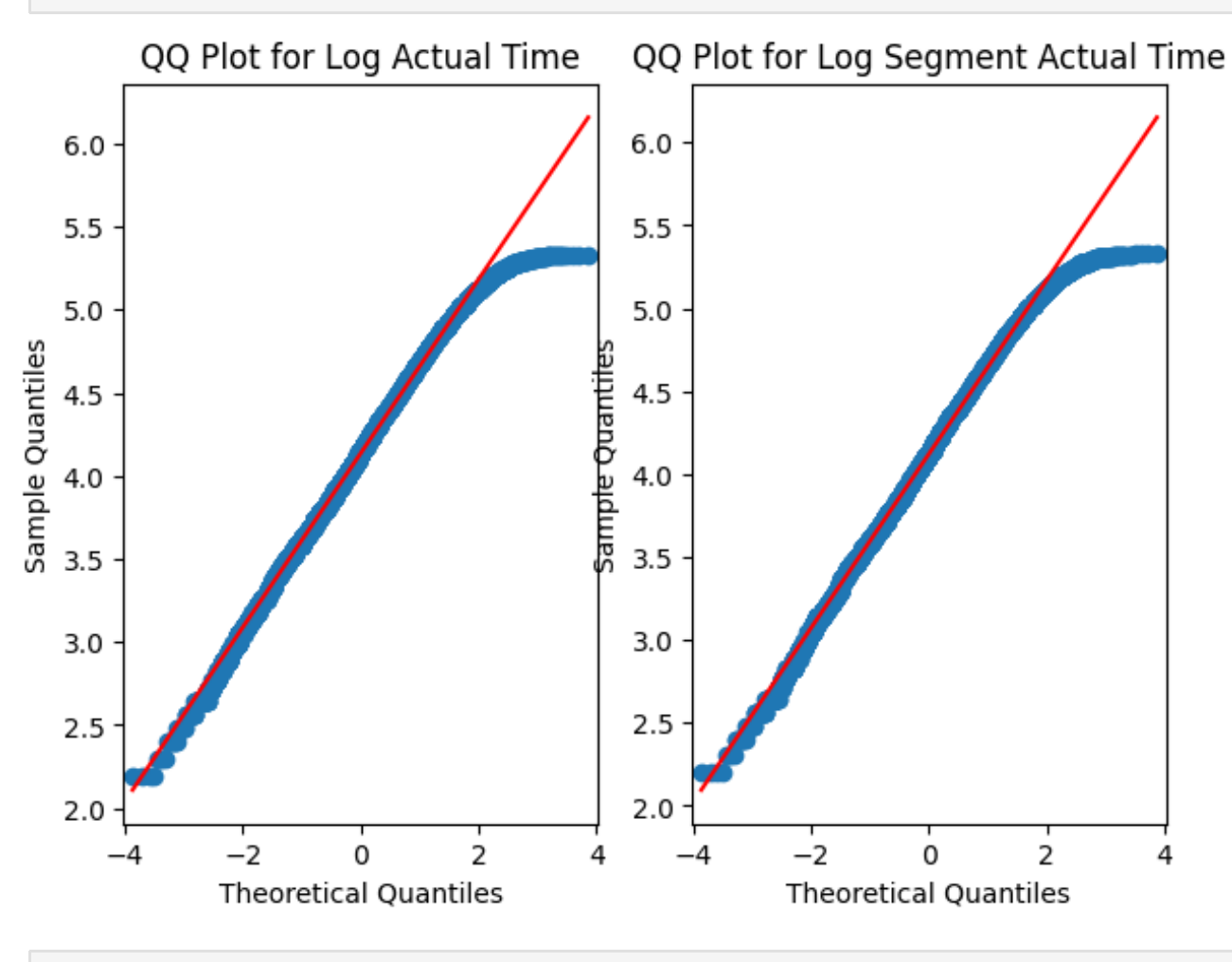
In [218…

```
# Lets peform ttest paired, however before that lets check for normality:


fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(modified_df['actual_time'],line='s',ax = axes[0])
axes[0].set_title("QQ Plot for Actual Time")

sm.qqplot(modified_df['segment_actual_time'],line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Segment Actual Time")

plt.show()
```



In [219…

```
# Since the data is not normal, hence using log transformation

x5 = np.log(modified_df['actual_time'])
x6 = np.log(modified_df['segment_actual_time'])

fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(x5,line='s',ax = axes[0])
axes[0].set_title("QQ Plot for Log Actual Time")

sm.qqplot(x6,line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Log Segment Actual Time")

plt.show()
```



In [220…

```
# Hypothesis Testing

H0 = "There is no difference between actual time and segment actual time"
H1 = "There is a significant difference between actual time and segment actual time"


stat,p_value = ttest_rel(x5,x6)

print(stat,p_value)

alpha = .05

if p_value<alpha:
  print("Rejecting Null Hypothesis")
  print(H1)
else:
  print("Fail to reject null hypothesis")
  print(H0)
```

```
116.08551564118632 0.0
Rejecting Null Hypothesis
There is a significant difference between actual time and segment actual time
```

In [ ]:

```
# Inferences:

# Statistically Significant Difference: A paired t-test revealed a highly significant difference (p-value = 0.0000)
# between actual_time and segment_actual_time. This means they are not the same, on average.

# Small Mean Discrepancy: On average, actual_time is approximately 0.71 minutes (about 42 seconds) longer than segment_actual_time.

# High Practical Significance: Despite the small per-trip difference, its cumulative impact is significant in logistics.
# This leads to substantial unaccounted-for operational time, increasing costs, disrupting schedules, and potentially impacting
# customer satisfaction and service level agreements.

# Signals Operational Gaps: This consistent discrepancy highlights underlying inefficiencies or inaccuracies in time
# measurement/aggregation, presenting a critical area for operational refinement and improved predictability.
```

Do hypothesis testing/ visual analysis between osrm distance aggregated value and segment osrm distance aggregated value

In [222…

```
# In this we are comparing below two features:

modified_df[['osrm_distance','segment_osrm_distance']]
```

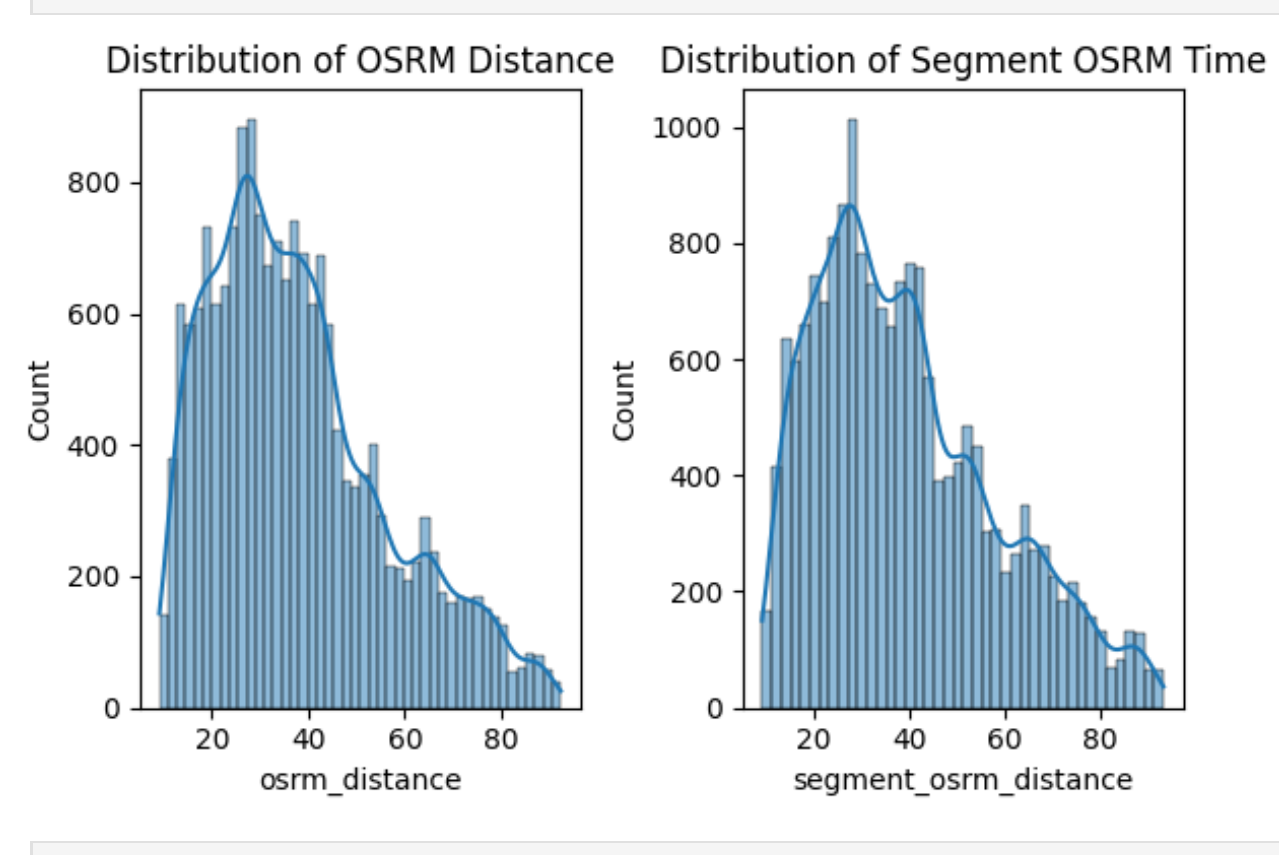
Out[222…

	osrm_distance	segment_osrm_distance
2	28.1994	28.1995
3	56.9116	55.9899
6	19.6800	19.8766
7	63.6461	63.6461
9	29.5696	29.5697
...	...	...
26362	25.7087	25.7087
26363	42.5213	42.1431
26364	40.6080	78.5869
26365	16.0185	16.0184
26367	28.0484	28.0484

18074 rows × 2 columns

In [225…

```
# Lets check vidually:

fig,axes = plt.subplots(1,2,figsize = (6,4))
axes = axes.flatten()

sns.histplot(modified_df['osrm_distance'],kde = True,ax = axes[0])
axes[0].set_title('Distribution of OSRM Distance')

sns.histplot(modified_df['segment_osrm_distance'],kde = True,ax = axes[1])
axes[1].set_title('Distribution of Segment OSRM Time')

plt.tight_layout()
plt.show()
```



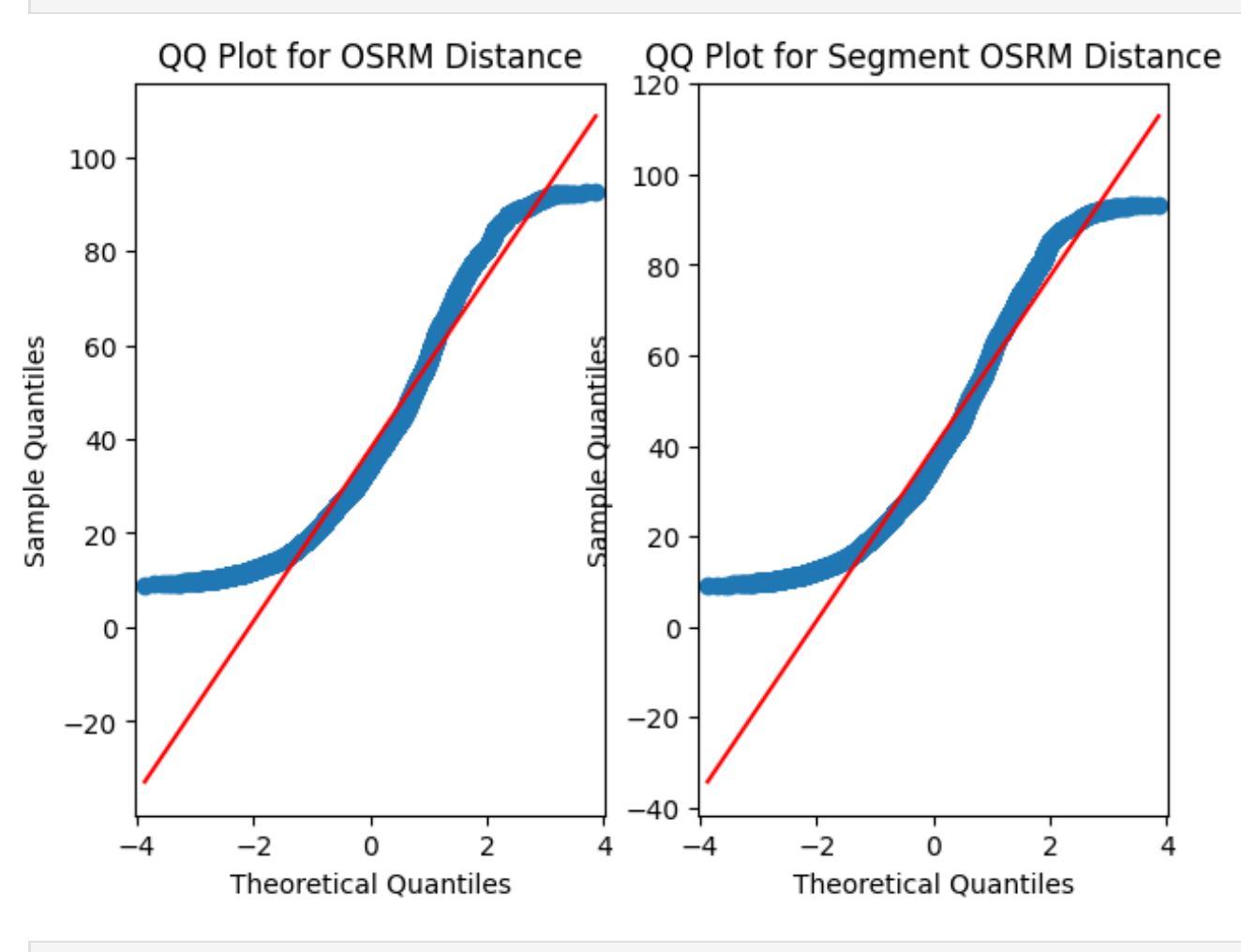
In [226…

```
# Vidually it looks similar, however lets do the statiscial summary and hypothesis testing

print('Descriptive Statistics for OSRM Distance:')
print(modified_df['osrm_distance'].describe())
print('\nDescriptive Statistics for segment_osrm_distance:')
print(modified_df['segment_osrm_distance'].describe())
```

```
Descriptive Statistics for OSRM Distance:
count    18074.000000
mean        37.894278
std         18.298163
min          9.072900
25%         24.037950
50%         34.667600
75%         48.288775
max         92.390900
Name: osrm_distance, dtype: float64

Descriptive Statistics for segment_osrm_distance:
count    18074.000000
mean        39.282341
std         19.045962
min          9.072900
25%         24.513400
50%         35.901850
75%         51.381675
max         93.350100
Name: segment_osrm_distance, dtype: float64
```

In [227…

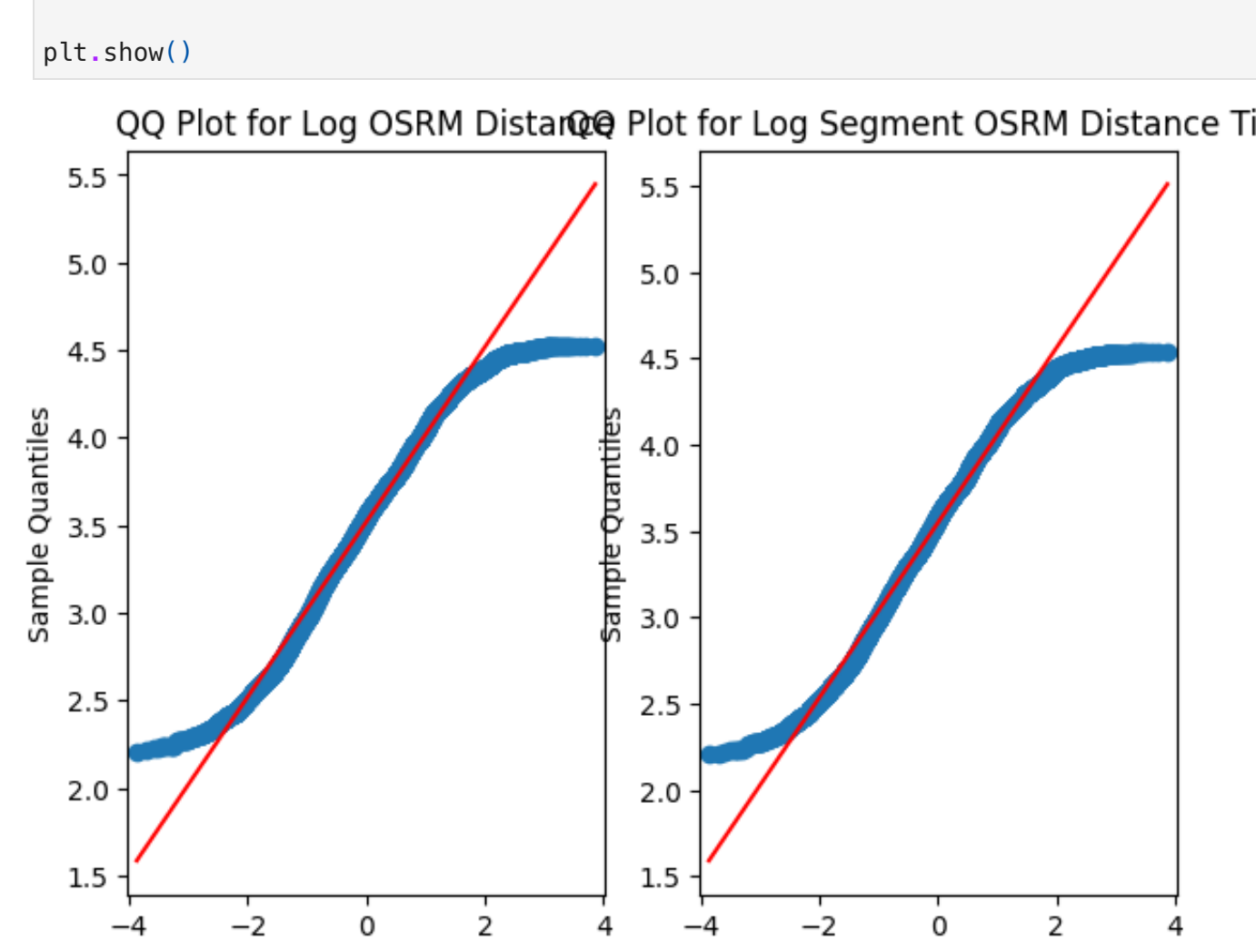
```
# Lets peform ttest paired, however before that lets check for normality:


fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(modified_df['osrm_distance'],line='s',ax = axes[0])
axes[0].set_title("QQ Plot for OSRM Distance")

sm.qqplot(modified_df['segment_osrm_distance'],line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Segment OSRM Distance")

plt.show()
```



In [229…

```
# Since the data is not normal, hence using log transformation

x7 = np.log(modified_df['osrm_distance'])
x8 = np.log(modified_df['segment_osrm_distance'])

fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(x7,line='s',ax = axes[0])
axes[0].set_title("QQ Plot for Log OSRM Distance")
```

```
sm.qqplot(x8,line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Log Segment OSRM Distance Time")

plt.show()
```



In [230…
```
# Hypothesis Testing

H0 = "There is no difference between osrm distance and segment osrm distance "
H1 = "There is a significant difference between osrm distance and segment osrm distance "


stat,p_value = ttest_rel(x7,x8)

print(stat,p_value)

alpha = .05

if p_value<alpha:
  print("Rejecting Null Hypothesis")
  print(H1)
else:
  print("Fail to reject null hypothesis")
  print(H0)
```

```
-41.10678804068895 0.0
Rejecting Null Hypothesis
There is a significant difference between osrm distance and segment osrm distance
```

In [ ]:
```
# Inferences

# Statistically Significant Difference: A paired t-test revealed a highly significant difference (p-value = 0.0)
# between osrm_distance and segment_osrm_distance, indicating these two metrics are not statistically identical.

# Similar Distributions, Minor Numerical Variance: Visually, both metrics exhibit similar right-skewed distributions,
# with closely aligned means and medians, yet show slight numerical differences.

# OSRM Inconsistency: This discrepancy suggests that the sum of OSRM distances for individual segments does not perfectly
# align with the OSRM distance calculated for the entire aggregated trip.

# Implication for Accuracy: The difference points to potential inaccuracies or varying calculation methodologies within the
# OSRM system when aggregating distances across segments versus for a complete origin-destination pair.
```

Do hypothesis testing/ visual analysis between osrm time aggregated value and segment osrm time aggregated value

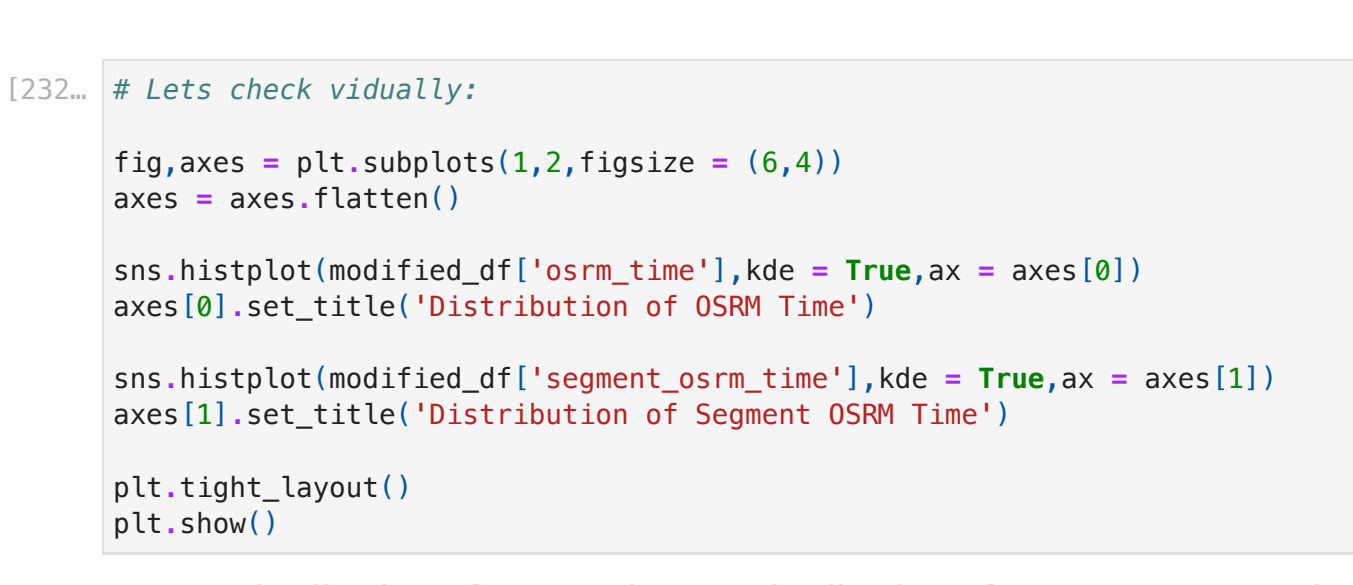
In [231…
```
# In this we are comparing below two features:

modified_df[['osrm_time','segment_osrm_time']]
```

Out[231…

	osrm_time	segment_osrm_time
2	26.0	26.0
3	42.0	39.0
6	15.0	16.0
7	46.0	45.0
9	27.0	26.0
...	...	...
26362	31.0	30.0
26363	41.0	42.0
26364	48.0	77.0
26365	14.0	14.0
26367	26.0	25.0

18074 rows × 2 columns

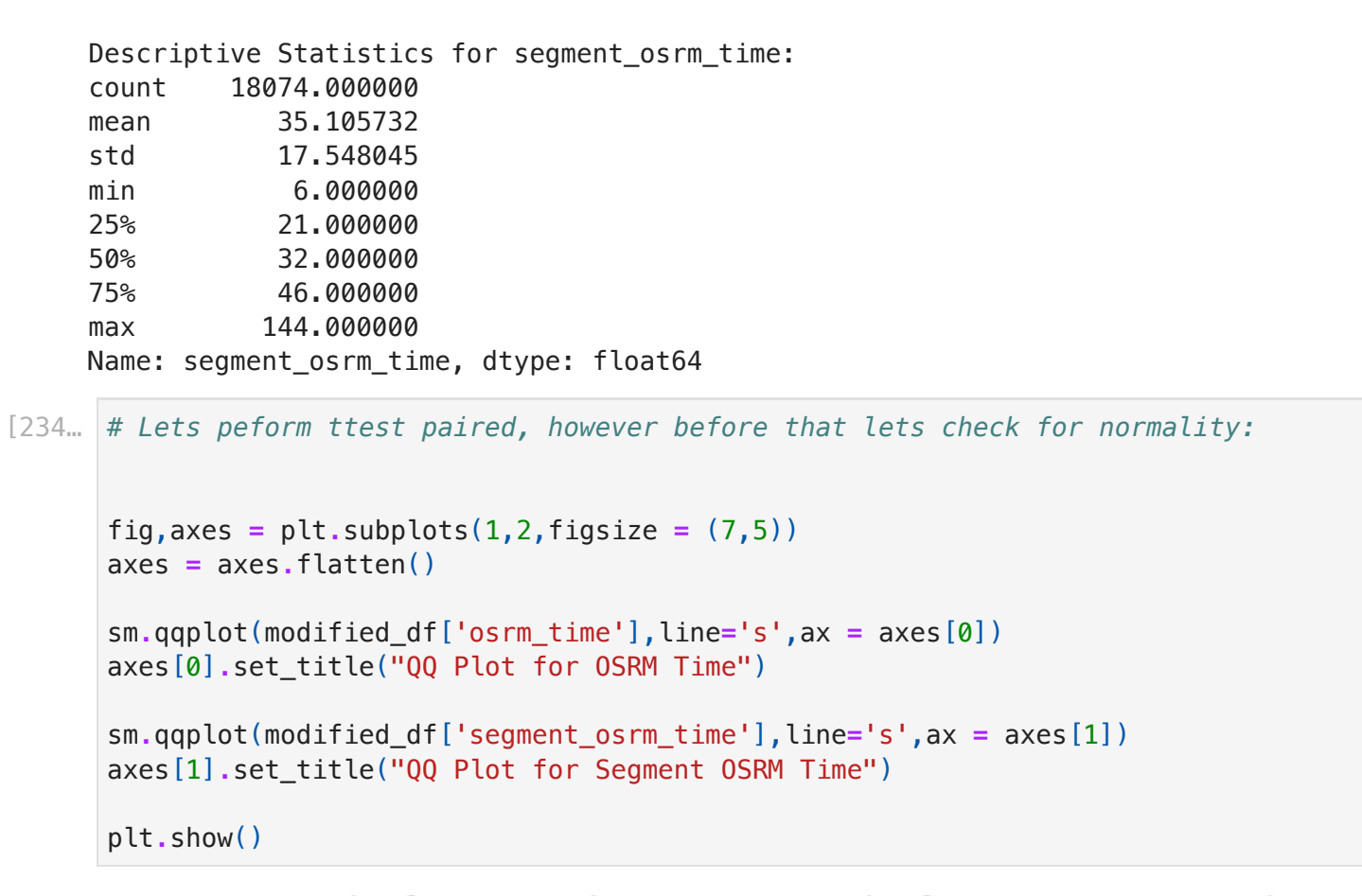
In [232…
```
# Lets check vidually:

fig,axes = plt.subplots(1,2,figsize = (6,4))
axes = axes.flatten()

sns.histplot(modified_df['osrm_time'],kde = True,ax = axes[0])
axes[0].set_title('Distribution of OSRM Time')

sns.histplot(modified_df['segment_osrm_time'],kde = True,ax = axes[1])
axes[1].set_title('Distribution of Segment OSRM Time')

plt.tight_layout()
plt.show()
```



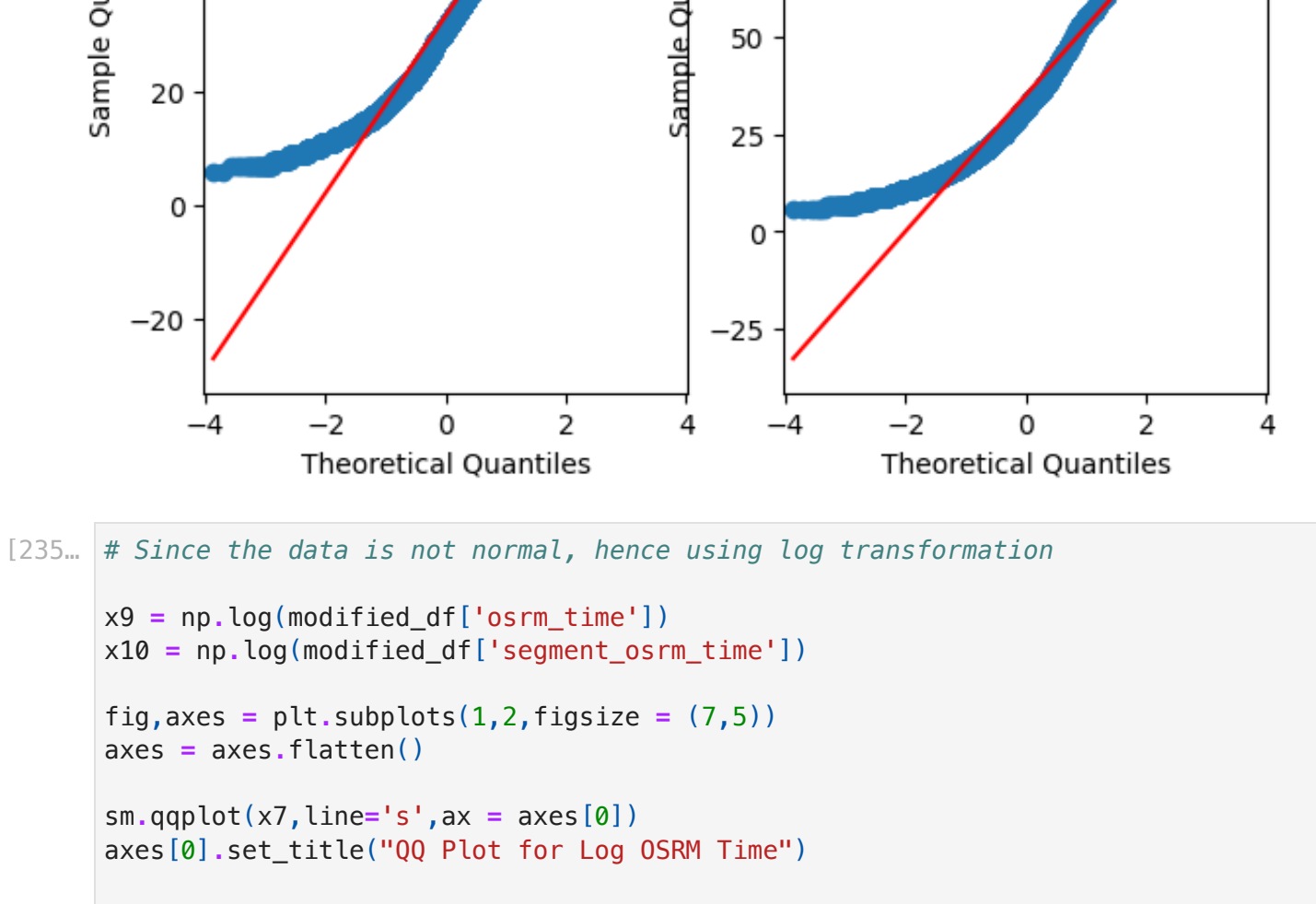
In [233…
```
# Vidually it looks similar, however lets do the statiscial summary and hypothesis testing

print('Descriptive Statistics for OSRM Time:')
print(modified_df['osrm_time'].describe())
print('\nDescriptive Statistics for segment_osrm_time:')
print(modified_df['segment_osrm_time'].describe())
```

```
Descriptive Statistics for OSRM Time:
count    18074.000000
mean        33.570931
std         15.713657
min          6.000000
25%         21.000000
50%         31.000000
75%         44.000000
max         89.000000
Name: osrm_time, dtype: float64

Descriptive Statistics for segment_osrm_time:
count    18074.000000
mean        35.105732
std         17.548045
min          6.000000
25%         21.000000
50%         32.000000
75%         46.000000
max        144.000000
Name: segment_osrm_time, dtype: float64
```

In [234…
```
# Lets peform ttest paired, however before that lets check for normality:

fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(modified_df['osrm_time'],line='s',ax = axes[0])
axes[0].set_title("QQ Plot for OSRM Time")

sm.qqplot(modified_df['segment_osrm_time'],line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Segment OSRM Time")

plt.show()
```



In [235…
```
# Since the data is not normal, hence using log transformation

x9 = np.log(modified_df['osrm_time'])
x10 = np.log(modified_df['segment_osrm_time'])

fig,axes = plt.subplots(1,2,figsize = (7,5))
axes = axes.flatten()

sm.qqplot(x7,line='s',ax = axes[0])
axes[0].set_title("QQ Plot for Log OSRM Time")

sm.qqplot(x8,line='s',ax = axes[1])
axes[1].set_title("QQ Plot for Log Segment OSRM Time")

plt.show()
```



In [236…
```
# Hypothesis Testing

H0 = "There is no difference between osrm time and segment osrm time "
H1 = "There is a significant difference between osrm time and segment osrm time "


stat,p_value = ttest_rel(x9,x10)

print(stat,p_value)

alpha = .05

if p_value<alpha:
  print("Rejecting Null Hypothesis")
  print(H1)
else:
  print("Fail to reject null hypothesis")
  print(H0)
```

```
-35.4688044384534 1.886688494109781e-266
Rejecting Null Hypothesis
There is a significant difference between osrm time and segment osrm time
```

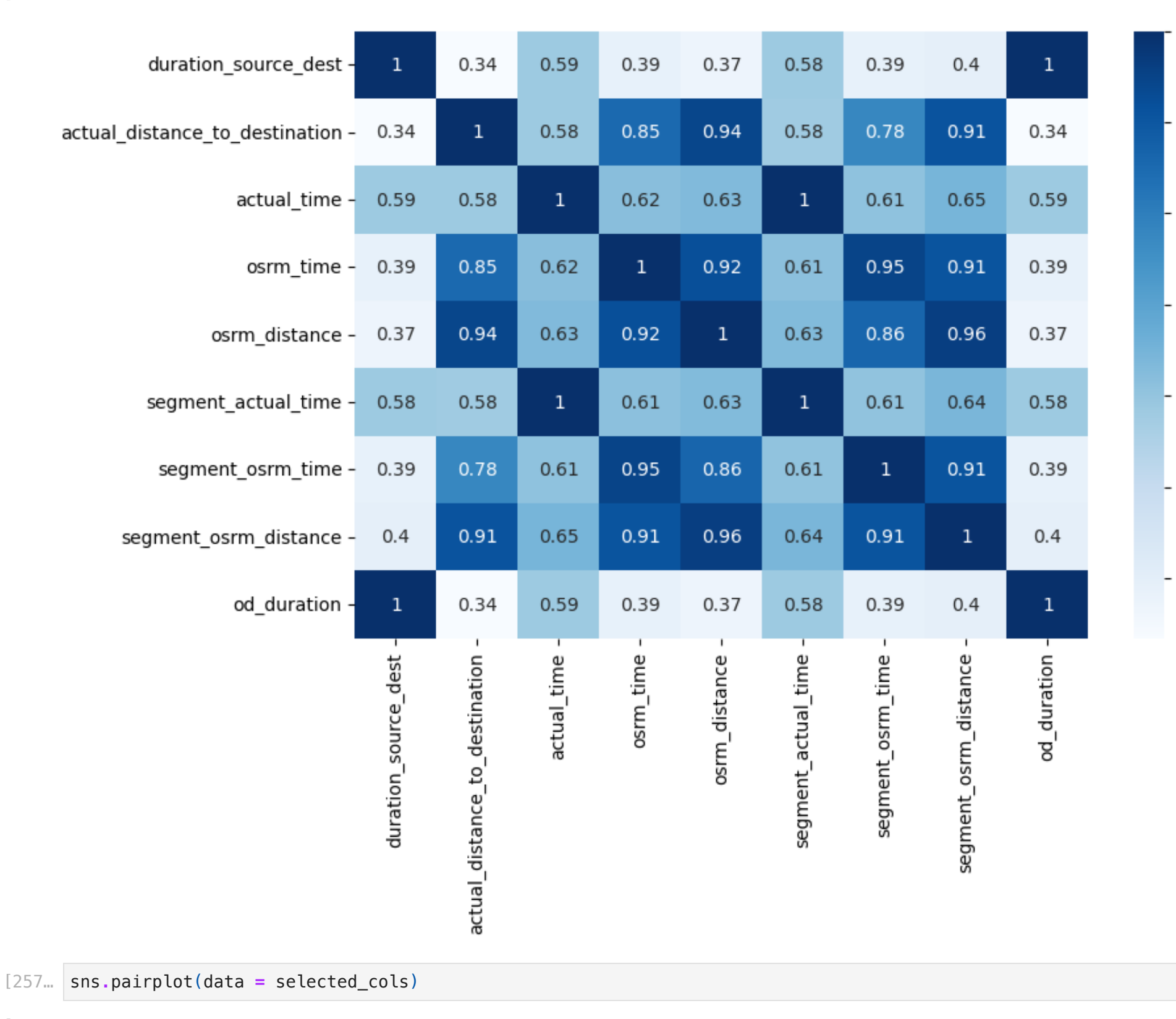
In [ ]:
```
# Inferences
# – Statistically Significant Difference: A paired t-test revealed a highly significant difference (p-value = 0.0)
# between osrm_time and segment_osrm_time, indicating these two time metrics are not statistically identical.

# – Similar Distributions, Minor Numerical Variance: Visually, both metrics exhibit similar right-skewed distributions,
# with closely aligned means and medians, yet show slight numerical differences.

# – OSRM Inconsistency: This discrepancy suggests that the sum of OSRM times for individual segments does not perfectly
# align with the OSRM time calculated for the entire aggregated trip.

# – Implication for Accuracy: The difference points to potential inaccuracies or varying calculation methodologies within
# the OSRM system when aggregating times across segments versus for a complete origin-destination pair.
```

Summary based on the hypothesis testing which we did for multiple features

- OSRM Significantly Underestimates Actual Delivery Times: On average, OSRM estimates are about 37 minutes shorter than actual delivery times, leading to unrealistic planning and customer expectations.
- Route Type Drives Delays: "Carting" operations experience greater and more variable delays compared to "FTL" (Full Truck Load), indicating OSRM struggles with multi-stop complexities.
- Distance is Not the Primary Delay Factor: The actual distance traveled has a weak correlation with time discrepancies; optimizing mileage alone won't solve predictability issues.
- Small Internal Time Discrepancies Accumulate: A statistically significant average difference of 0.71 minutes (42 seconds) between actual_time and segment_actual_time accumulates to substantial hidden operational time and costs over many trips.
- OSRM Has Internal Inconsistencies: There are statistically significant differences between aggregated OSRM metrics and the sum of their segment-level counterparts, questioning the accuracy of OSRM at different granularities.
- Action Needed on Operational Bottlenecks: Investigate non-distance factors (e.g., loading/unloading, traffic patterns, specific geographies) causing OSRM underestimation, especially for Carting routes.
- Enhance Planning Accuracy with Tailored Models: Develop specific forecasting models for FTL and Carting, incorporating granular features and possibly fine-tuning OSRM with historical data.
- Quantify & Manage Hidden Costs: Calculate the financial impact of all observed time discrepancies (fuel, labor, vehicle wear) to prioritize improvement efforts and optimize resource allocation.
- Improve Data Quality & Consistency: Implement programs to ensure accurate and consistent recording of actual times and address internal OSRM inconsistencies to improve data reliability for planning.
- Drive Continuous Improvement: Explore advanced routing solutions, pilot new technologies, and establish feedback loops to continuously refine models and processes for better predictability and efficiency.

In [256…
```
# Select only numeric columns for correlation calculation
numeric_df = modified_df.select_dtypes(include=np.number)



selected_cols = modified_df[['duration_source_dest','actual_distance_to_destination','actual_time','osrm_time','osrm_distance',
                             'segment_actual_time','segment_osrm_time','segment_osrm_distance','od_duration']]


plt.figure(figsize=(9, 6))
sns.heatmap(selected_cols.corr(), annot=True,cmap='Blues')
```

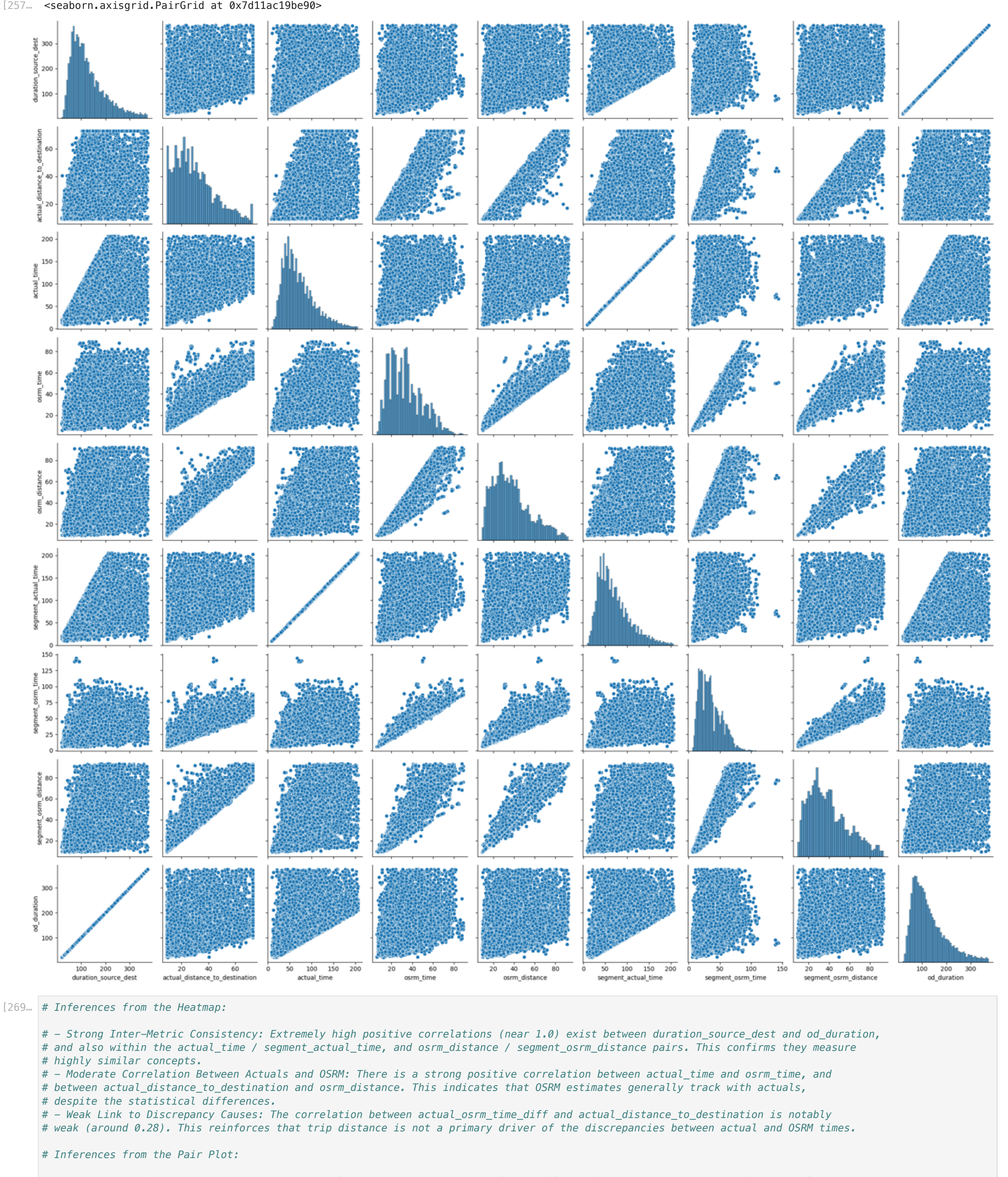
Out[256… <Axes: >



	duration_source_dest	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_distance	od_duration
duration_source_dest	1	0.34	0.59	0.39	0.37	0.58	0.39	0.4	1
actual_distance_to_destination	0.34	1	0.58	0.85	0.94	0.58	0.78	0.91	0.34
actual_time	0.59	0.58	1	0.62	0.63	1	0.61	0.65	0.59
osrm_time	0.39	0.85	0.62	1	0.92	0.61	0.95	0.91	0.39
osrm_distance	0.37	0.94	0.63	0.92	1	0.63	0.86	0.96	0.37
segment_actual_time	0.58	0.58	1	0.61	0.63	1	0.61	0.64	0.58
segment_osrm_time	0.39	0.78	0.61	0.95	0.86	0.61	1	0.91	0.39
segment_osrm_distance	0.4	0.91	0.65	0.91	0.96	0.64	0.91	1	0.4
od_duration	1	0.34	0.59	0.39	0.37	0.58	0.39	0.4	1

In [257…
```
sns.pairplot(data = selected_cols)
```

Out[257… <seaborn.axisgrid.PairGrid at 0x7d11ac19be90>

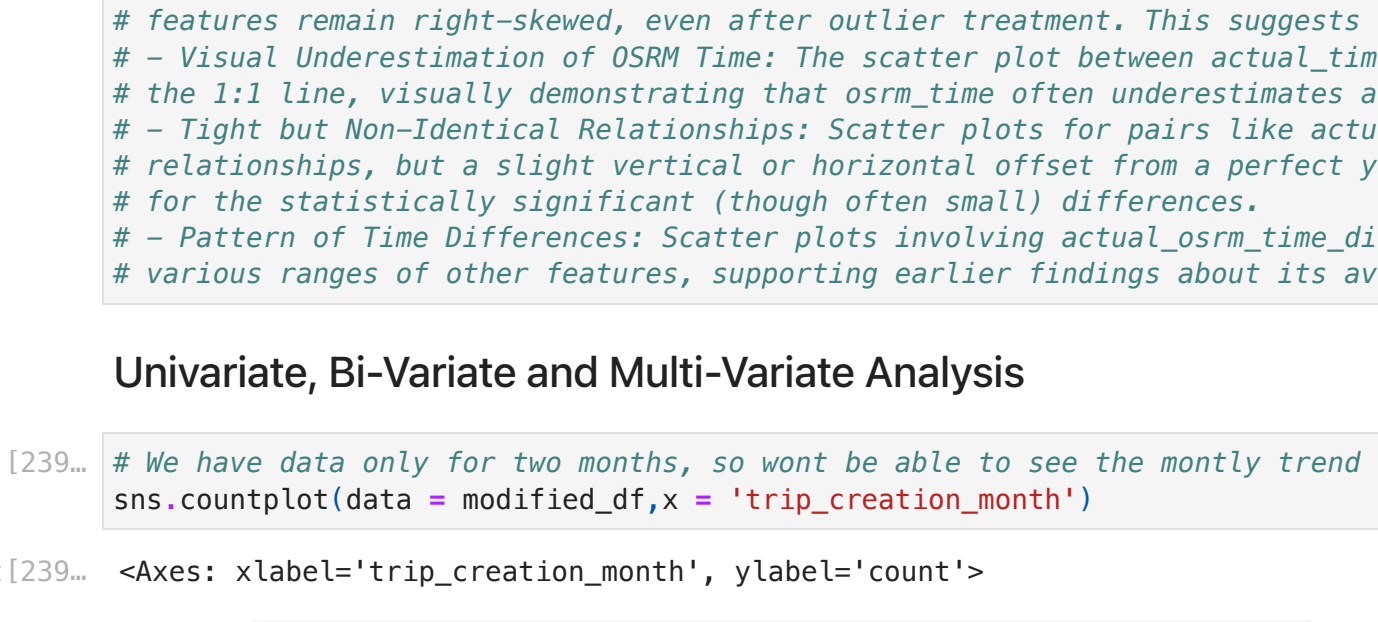


In [269…
```
# Inferences from the Heatmap:

# – Strong Inter-Metric Consistency: Extremely high positive correlations (near 1.0) exist between duration_source_dest and od_duration,
# and also within the actual_time / segment_actual_time, and osrm_distance / segment_osrm_distance pairs. This confirms they measure
# highly similar concepts.
# – Moderate Correlation Between Actuals and OSRM: There is a strong positive correlation between actual_time and osrm_time, and
# between actual_distance_to_destination and osrm_distance. This indicates that OSRM estimates generally track with actuals,
# despite the statistical differences.
# – Weak Link to Discrepancy Causes: The correlation between actual_osrm_time_diff and actual_distance_to_destination is notably
# weak (around 0.28). This reinforces that trip distance is not a primary driver of the discrepancies between actual and OSRM times.

# Inferences from the Pair Plot:

# – Persistent Skewed Distributions: The diagonal histograms in the pair plot visibly confirm that most time and distance-related
# features remain right-skewed, even after outlier treatment. This suggests natural distribution for these metrics.
# – Visual Underestimation of OSRM Time: The scatter plot between actual_time and osrm_time typically shows points clustering above
# the 1:1 line, visually demonstrating that osrm_time often underestimates actual_time.
# – Tight but Non-Identical Relationships: Scatter plots for pairs like actual_time vs. segment_actual_time show very tight, linear
# relationships, but a slight vertical or horizontal offset from a perfect y=x line can often be observed, providing a visual cue
# for the statistically significant (though often small) differences.
# – Pattern of Time Differences: Scatter plots involving actual_osrm_time_diff might visually hint at its distribution across
# various ranges of other features, supporting earlier findings about its average value and spread.
```

Univariate, Bi-Variate and Multi-Variate Analysis

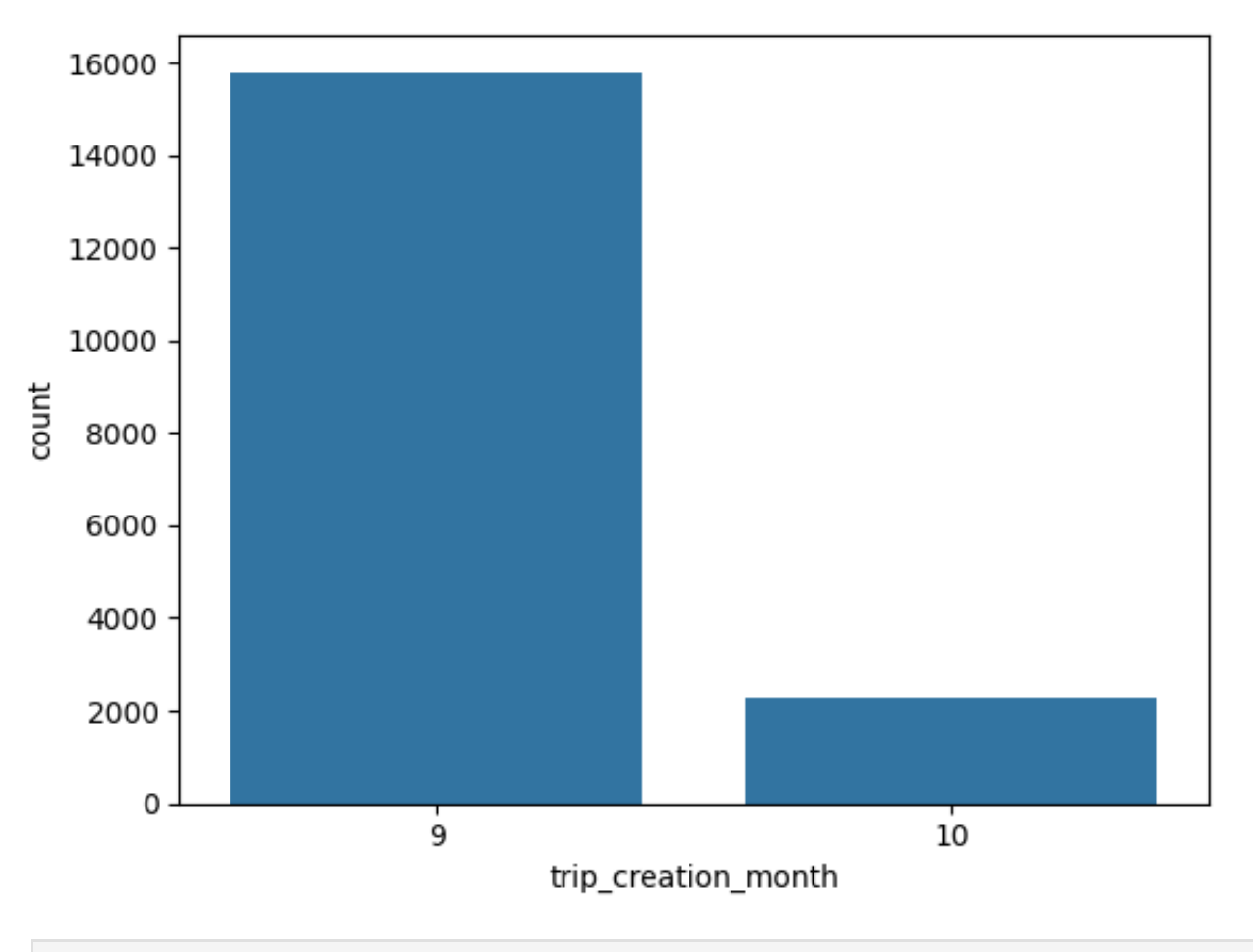
In [239…
```
# We have data only for two months, so wont be able to see the montly trend
sns.countplot(data = modified_df,x = 'trip_creation_month')
```

Out[239… <Axes: xlabel='trip_creation_month', ylabel='count'>



In [244…
```
# Let look at the distribution for 9th month only on a daily basis

daily_count = modified_df[modified_df['trip_creation_month'] == 9]

sns.countplot(data = daily_count,x = 'trip_creation_day')
```

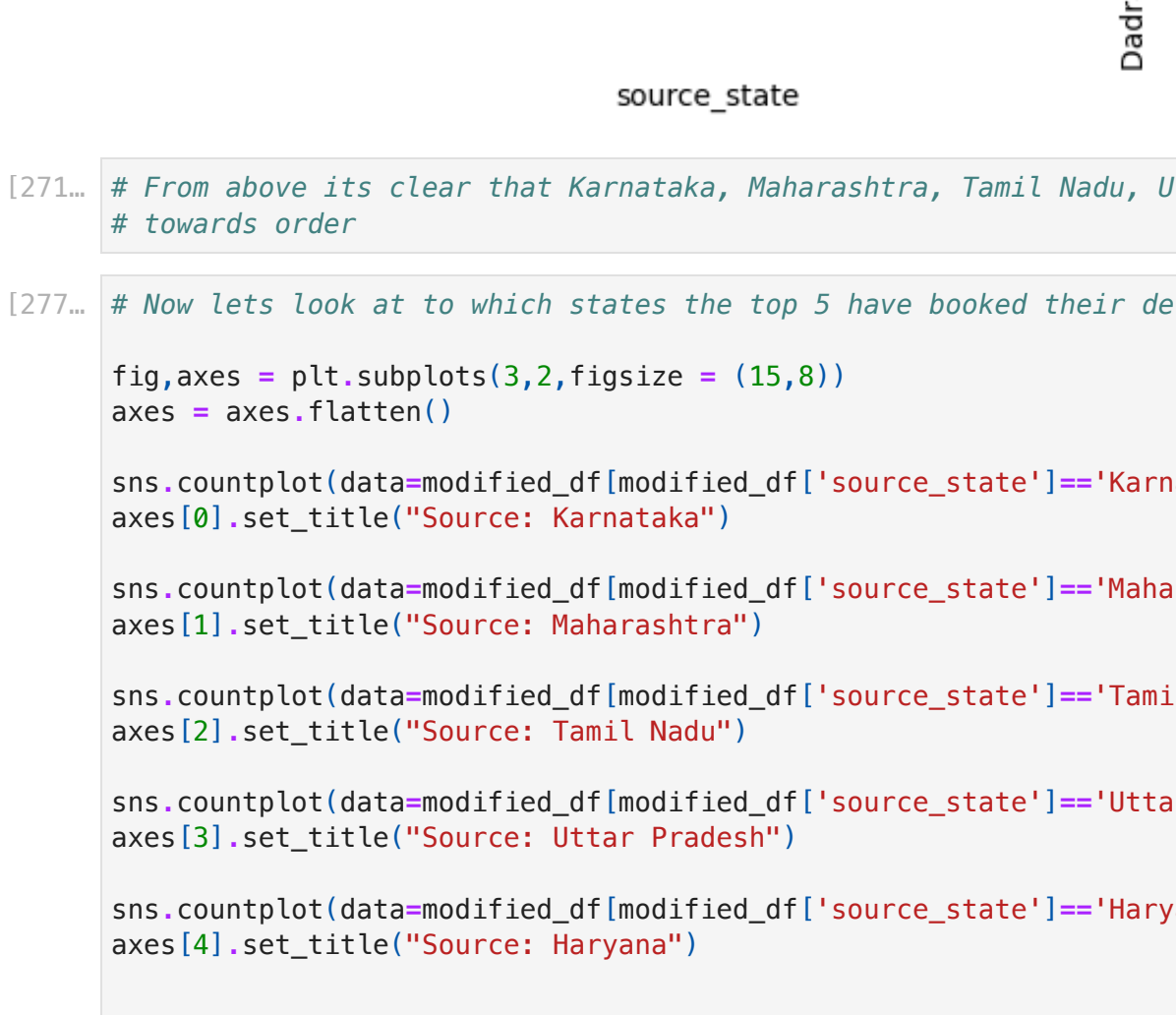
Out[244… <Axes: xlabel='trip_creation_day', ylabel='count'>



In [ ]:
```
# Here also the data is not available for entire 30 days, so we have very less data to infer
```

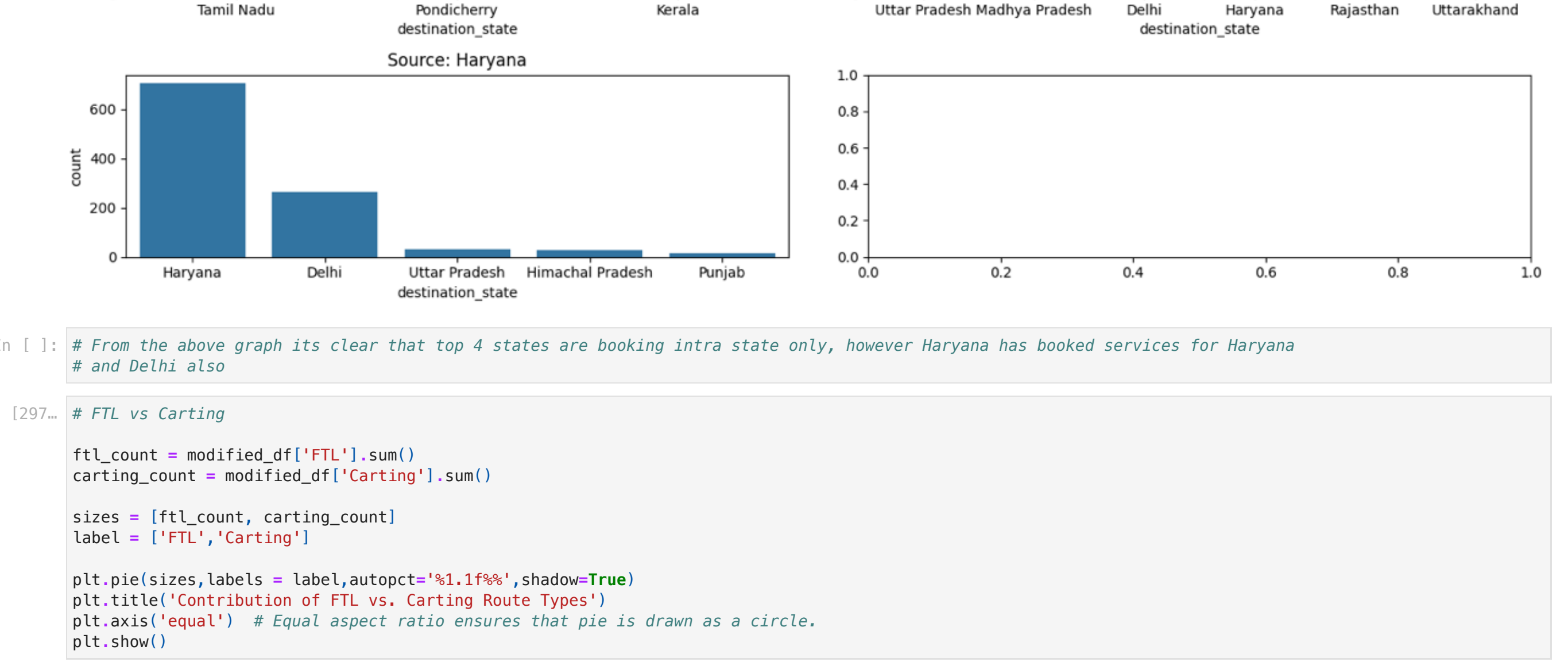
In [270…
```
# Look at the top states who are providing maximum orders

modified_df['source_state'].value_counts().plot(kind='bar')
```

Out[270… <Axes: xlabel='source_state'>



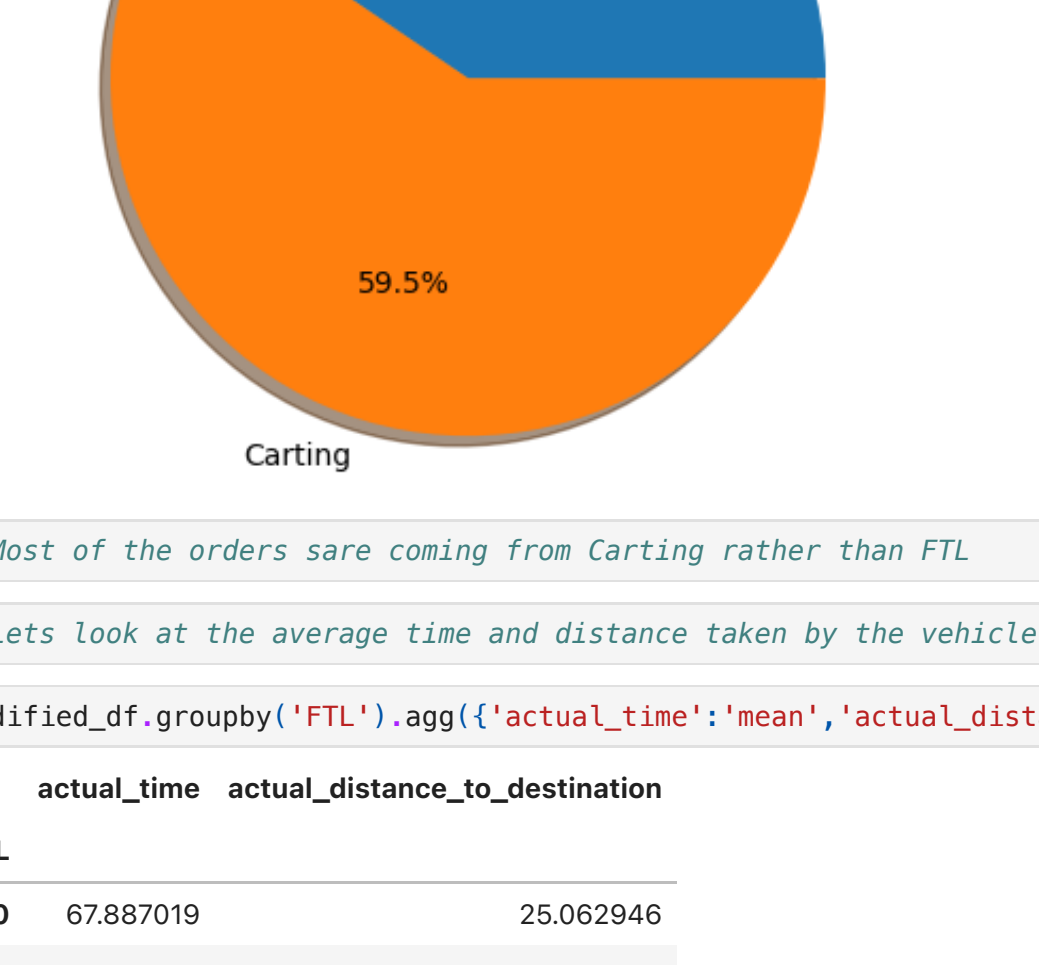
In [271…
```
# From above its clear that Karnataka, Maharashtra, Tamil Nadu, Uttar Pradesh and Haryana are top 5 states with highest contribution
# towards order
```

In [277…
```
# Now lets look at to which states the top 5 have booked their delivery

fig,axes = plt.subplots(3,2,figsize = (15,8))
axes = axes.flatten()

sns.countplot(data=modified_df[modified_df['source_state']=='Karnataka'],x = 'destination_state',ax=axes[0])
axes[0].set_title("Source: Karnataka")

sns.countplot(data=modified_df[modified_df['source_state']=='Maharashtra'],x = 'destination_state',ax=axes[1])
axes[1].set_title("Source: Maharashtra")

sns.countplot(data=modified_df[modified_df['source_state']=='Tamil Nadu'],x = 'destination_state',ax=axes[2])
axes[2].set_title("Source: Tamil Nadu")

sns.countplot(data=modified_df[modified_df['source_state']=='Uttar Pradesh'],x = 'destination_state',ax=axes[3])
axes[3].set_title("Source: Uttar Pradesh")

sns.countplot(data=modified_df[modified_df['source_state']=='Haryana'],x = 'destination_state',ax=axes[4])
axes[4].set_title("Source: Haryana")


plt.tight_layout()
plt.show()
```



In [ ]:
```
# From the above graph its clear that top 4 states are booking intra state only, however Haryana has booked services for Haryana
# and Delhi also
```

In [297…
```
# FTL vs Carting

ftl_count = modified_df['FTL'].sum()
carting_count = modified_df['Carting'].sum()

sizes = [ftl_count, carting_count]
label = ['FTL','Carting']

plt.pie(sizes,labels = label,autopct='%1.1f%%',shadow=True)
plt.title('Contribution of FTL vs. Carting Route Types')
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```



In [ ]:
```
# Most of the orders sare coming from Carting rather than FTL
```

In [ ]:
```
# Lets look at the average time and distance taken by the vehicle under these categories
```

In [302…
```
modified_df.groupby('FTL').agg({'actual_time':'mean','actual_distance_to_destination':'mean'})
```

Out[302…

	actual_time	actual_distance_to_destination
FTL		
0	67.887019	25.062946
1	76.200273	37.423929

In [303…
```
modified_df.groupby('Carting').agg({'actual_time':'mean','actual_distance_to_destination':'mean'})
```

Out[303…

	actual_time	actual_distance_to_destination
Carting		
0	76.200273	37.423929
1	67.887019	25.062946

In [ ]:
```
# Its is clear that FTL covers more distance than Carting and hence taking more time to deliver the goods,
# however the difference is not much significant than that of carting, so if more emphasis can be given on Carting
```

Final Business Insights and Recmmendations

Business Insights

In [ ]:
```
# – Karnataka, Maharashtra, Tamil Nadu, Uttar Pradesh and Haryana are top 5 states with highest contribution towards order
# – Top 4 states are booking intra state only, however Haryana has booked services for Haryana and Delhi also
# – Most of the orders sare coming from Carting (~60%) with an average time of 67 mins and distance of 27, where as for FTL these
# numbers stands at 76 mins and 37
# –  Current Time Estimates Are Consistently Off: On average, open source estimated delivery times (OSRM time) are significantly
# shorter—by about 37 minutes per trip—than the actual time it takes. This shows that open source data is not reliable.
# – Multi-Stop Deliveries Are the Biggest Problem: Trips with multiple stops (Carting) show much larger and more unpredictable delays
# compared to direct FTL (Full Truck Load) trips. The Open Source Routing is not suitable to handle the complexities of multi-stop
# routes well.
# – Distance Isn't the Only Culprit: Simply focusing on finding the shortest routes won't fix our delay issues. The actual distance
# traveled has a surprisingly weak connection to why our estimates are wrong. Other factors are more influential.
# – Small Delays Accumulate to Big Problems: Even tiny discrepancies (averaging 42 seconds per trip) between reported actual time
# and the sum of segment actual times add up significantly. Over many trips, these 'hidden' minutes can translate to substantial
# wasted operational hours and increased costs.
# – Our Internal Data Shows Inconsistencies: There are measurable differences within our OSRM data itself—the total estimated
# distance/time for a trip doesn't perfectly match the sum of its segment parts. This indicates a foundational data issue affecting
# planning accuracy.
# – Impact on Efficiency and Customer Trust: These estimation gaps and data inconsistencies directly reduce our operational
# efficiency, inflate costs (e.g., fuel, overtime), and can negatively impact customer commitments and trust.
```

Recommendations

In [304…
```
# – Investigate 'Why' Beyond Distance: Conduct deeper dives into non-distance factors causing delays, such as specific traffic hotspots,
# loading/unloading processes, driver practices, and regional challenges. This will pinpoint exact areas for improvement.
# – Prioritize Multi-Stop Route Optimization: Implement specialized tools or refined processes for Carting operations. This could
# involve dynamic routing that accounts for real-time conditions and optimized stop sequencing.
# – Improve Time Data Accuracy: Enhance our methods for recording and aggregating time at each stage of a trip. Ensuring that
# segment actual times accurately reflect overall actual time will close a critical 'hidden cost' gap.
#  – Refine Estimation Models: Work towards customizing our OSRM parameters with our historical performance data. Developing
#  tailored prediction models for FTL vs. Carting will yield much more accurate delivery estimates.
# – Quantify Financial Impact: Calculate the true monetary cost of these time discrepancies (e.g., in fuel, labor, maintenance).
# This will help prioritize and justify investments in solutions.
# – Set Realistic Expectations: Update internal and external stakeholders with more accurate estimated delivery times, especially
# for complex routes, to manage expectations and improve customer satisfaction.
```